Multi-Agent Tool-Integrated Policy Optimization with Process Reward

Zhanfeng Mo [*1] **Xiaogang Xu** [*1] **Xingxuan Li** [*1] **Lidong Bing** [1]

Abstract

Large language models (LLMs) increasingly rely on multi-turn tool-integrated planning (TIP) to solve open-ended, knowledge-intensive, and complex reasoning tasks. Existing single-agent TIP frameworks suffer from limited context length and noisy tool responses, motivating the use of multi-agent frameworks with planner- and worker-agents to manage context. However, effective reinforcement learning (RL) post-training methods for tool-integrated multi-agent frameworks remain underexplored. To address this gap, we propose Multi-Agent Tool-Integrated Policy Optimization (**MATPO**), a principled policy gradient method that enables distinct roles (planner and worker) to be trained within a single LLM instance using role-specific prompts. Furthermore, we develop **MATPO-PR** (MATPO with Process Reward) to tackle the credit assignment problem in multi-turn planner-worker interactions by optimizing the quality of each intermediate worker-agent response. Our experiments show that MATPO-PR consistently outperforms single-agent baselines in both performance and robustness to noisy tool responses on three deep research benchmarks. Our findings highlight the effectiveness of unifying multiple agent roles within a single LLM and provide practical insights for effective RL training with multi-agent TIP systems. Our code is available at https://github.com/xiaogang00/MATPO-PR

1. Introduction

Tool-augmented LLM agents have revolutionized workflows across domains by enabling LLMs to solve complex problems through external tool use (Dong et al., 2025; Qian et al., 2025; Qu et al., 2025; Li, 2024; Singh et al., 2025; Chen et al., 2025a). These agents operate via multi-turn tool-integrated planning (TIP): after receiving a user query, an agent performs multiple iterations of planning-and-tool-calling until it is confident enough to produce a final answer.

[*]Equal contribution [1]MiroMind AI, Singapore. Correspondence to: Lidong Bing <lidong.bing@shanda.com>.

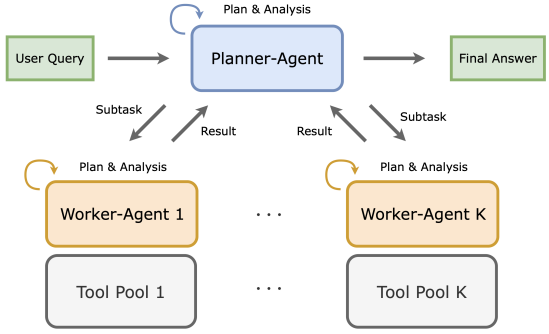


Figure 1. Multi-agent TIP framework. The planner-agent decomposes queries into subtasks and delegates them to worker-agents, which execute via single-agent TIP and return results. The planner then generates successive subtasks or the final answer.

Among the available tools, search APIs are particularly crucial, allowing LLMs to access external knowledge far beyond their parametric memory. Current implementations typically employ a single-agent TIP paradigm (Dong et al., 2026; Jin et al., 2025), where one LLM instance handles the entire planning and tool-calling process. However, this approach faces two key limitations: 1. **High token cost**: tool responses (*e.g.*, web search results) often consume excessive tokens, making long-horizon multi-turn TIP infeasible when the LLM's context window is limited; 2. **Contextual distraction**: tool responses often contain irregular patterns (*e.g.*, URLs) or information irrelevant to the query, which can distract the LLM's attention and impair its ability to plan high-quality subsequent actions.

These limitations motivate the **multi-agent** TIP paradigm (Hu et al., 2025; Du et al., 2024), which amortizes the TIP process across multiple agent roles: a planner-agent that decomposes queries into subtasks, and worker-agents that solve these subtasks via single-agent TIP (Figure 1). This design alleviates the high-token-cost issue by decomposing the full rollout into shorter, manageable segments. Moreover, tool-response tokens are localized within each worker's context and do not affect the planner's subsequent reasoning. Importantly, these agent roles can be instantiated within a *single* LLM by managing role-specific system prompts, avoiding the need for multiple model deployments. While multi-agent TIP offers clear benefits in saving token costs and mitigating contextual distractions, training such systems

introduces new challenges, particularly when each agent operates on separate model instances, which incurs infrastructure overhead due to uneven workloads and increased parameter consumption. This naturally raises the following research questions: 1. How can we perform multi-agent RL training effectively using a single LLM? 2. How should credit assignment be handled in multi-turn planner-worker interactions? 3. Can the planner- and worker-agent roles coexist and be trained within a single LLM instance?

In this paper, we provide affirmative answers to these questions. We propose Multi-Agent Tool-Integrated Policy Optimization (**MATPO**), a principled RL algorithm for the **multi-agent-in-one-model** setting, where a single LLM serves as both planner and worker through role-specific system prompts. MATPO derives the policy gradient for nested planner–worker rollouts and introduces a $G\times(T+1)$ group-relative normalization that shares the outcome reward across planner and worker sub-rollouts, building upon recent advances in RL post-training (Shao et al., 2024; Noukhovitch et al., 2025; Zhong et al., 2025). It can be implemented on top of existing single-agent RL frameworks (*e.g.,* veRL[1]) with minimal modification. In practice, MATPO exhibits more stable learning curves than single-agent approaches.

However, in long-horizon planning, producing a correct final answer depends on a sequence of valid intermediate subtasks, whereas MATPO relies on a sparse outcome reward that assigns a single signal to the whole rollout. To provide turn-level supervision, we introduce **MATPO-PR**, which integrates process rewards into the MATPO framework. Inspired by recent advances in process reward models (Lightman et al., 2024; Wang et al., 2024; Khalifa et al., 2025) and LLM-as-judge verification (Li et al., 2025a), MATPO-PR employs an LLM judge to score each intermediate planner–worker interaction and constructs discounted process returns that credit effective intermediate actions, encouraging the agent to reach correct solutions earlier and more reliably.

Contributions. 1. We present MATPO, a principled RL post-training method for the **multi-agent-in-one-model** setting, with a policy-gradient derivation for nested planner–worker rollouts and a $G\times(T+1)$ group-relative normalization across planner and worker sub-rollouts. 2. We introduce MATPO-PR, a process-reward variant that provides turn-level supervision across planner–worker interactions, enabling temporal credit assignment in long-horizon rollouts. 3. MATPO and MATPO-PR are implemented on top of standard single-agent RL frameworks via a nested-rollout and packed-batch design, with minimal modification to the training loop. 4. Experiments on three deep research benchmarks show consistent improvements over single-agent baselines, with MATPO-PR further improving over MATPO.

[1] https://github.com/volcengine/verl

2. Related Work

2.1. Tool-Integrated Agent Frameworks

Tool-integrated planning (TIP) has emerged as a crucial paradigm for LLMs to tackle complex, knowledge-intensive tasks through iterative reasoning with external tools (Zhao et al., 2023; Li et al., 2024; Xu & Peng, 2025; OpenAI, 2025). Representative single-agent approaches include function-calling LLMs (Yang et al., 2025; Qin et al., 2025), ReAct-style agents (Yao et al., 2023; Li et al., 2025d;b; Tao et al., 2025), structured workflow agents (Team et al., 2025), and web agents (Zhou et al., 2024a). However, single-agent TIP faces fundamental challenges: 1. the LLM's context window is quickly saturated by lengthy tool responses, hindering scalability to deeper reasoning chains (Zhang et al., 2025; Jin et al., 2024; Xiao et al., 2024); 2. noisy tool outputs can disrupt reasoning and induce cascading errors (Zhou et al., 2024c). To mitigate these issues, recent work explores multi-agent frameworks (Hu et al., 2025; MiroMind, 2025b; Du et al., 2024; Motwani et al., 2025; **?**) where planner- and worker-agents collaborate: the planner decomposes tasks and delegates subtasks to workers, containing noisy outputs within worker contexts. Training methodologies for multi-agent LLM systems have also been studied recently, including M-GRPO (**?**) and ReMA (**?**). Liu et al. (2025a) introduces training for multi-turn multi-agent zero-sum games, but does not address the TIP setting.

2.2. Tool-integrated Agentic Reinforcement Learning

Reinforcement learning with verifiable rewards (RLVR) has proven effective for single-agent TIP (Shao et al., 2024; Jin et al., 2025; **?**; MiroMind, 2025a; Nguyen et al., 2025), with recent advances introducing more efficient algorithms such as asynchronous RLHF (Noukhovitch et al., 2025) and token-level optimization (Zhong et al., 2025). Trajectory filtering has been explored for math problem solving with code (Li et al., 2025e; Xue et al., 2025; Feng et al., 2025) and GUI tasks (Dong et al., 2026). Another line of work combines SFT or DPO (Rafailov et al., 2023) on cold-start trajectories with RLVR (Li et al., 2025b; Tao et al., 2025; Wei et al., 2025; Ouyang et al., 2025; Li et al., 2025c; MiroMind, 2025a). A key challenge in multi-turn RL is credit assignment across interaction steps. Hierarchical approaches like ArCHer (Zhou et al., 2024b) address this through actor-critic frameworks, while process reward models (Lightman et al., 2024; Wang et al., 2024; Khalifa et al., 2025) provide step-level supervision for reasoning tasks. LLM-as-judge methods (Li et al., 2025a; Liu et al., 2025b) offer scalable alternatives to human annotation. Self-improvement through multi-agent interaction (Qu et al., 2024; Chen et al., 2025b) has also shown promise. While these methods show gains in single-agent settings, principled extensions to the multi-agent-in-one-model regime remain underexplored.

3. Problem Setup

3.1. Single-Agent Multi-Turn Reinforcement Learning

We first recap single-agent multi-turn RL before extending to the multi-agent setting. Let $\pi_\theta(\cdot|\cdot)$ be an LLM parameterized by θ. For each query $q \sim \mathcal{D}$, the agent aims to generate the correct answer via multi-turn TIP (Figure 2).

Recent works (Dong et al., 2025; Qian et al., 2025) have shown that RLVR is a promising approach for optimizing LLMs' ability to perform the multi-turn TIP process. Suppose $r(\cdot)$ is a reward function that assigns 1 to correct answers and 0 to incorrect ones. The RL objective of the single-agent TIP system is

$$\min_\theta J(\pi_\theta) \triangleq \mathbb{E}_{q\sim\mathcal{D},\tau\sim\pi_\theta}[r(\tau)],\ \tau \triangleq [a_1, s_1, .., a_T],$$
$$a_t \sim \pi_\theta(\cdot|[p_{\text{sys}}, q, a_1, s_1, ..., s_{t-1}]),\ s_t \sim \text{Tool}(a_t).$$

Specifically, p_{sys} is the system prompt defining the agent role and tool schema, a_t is the LLM-generated action at turn t including planning and tool-call blocks, $\text{Tool}(\cdot|a_t)$ is the invoked tool conditioned on a_t, s_t is its response, and τ denotes the complete TIP rollout trajectory.

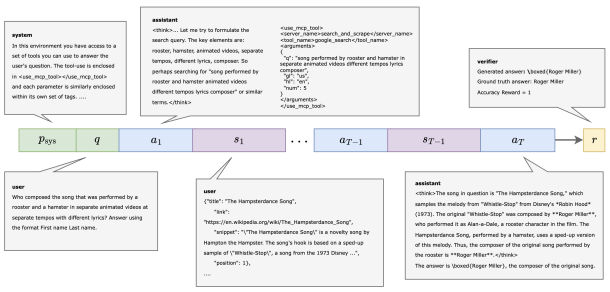


Figure 2. Visualization of a single-agent multi-turn TIP rollout. The LLM solves a query through iterative planning and tool-use. At each step, it plans a tool call, executes it with the parsed parameters, and uses the tool response to decide the next move, continuing until it is confident enough to produce a final answer.

3.2. Single-Agent Group Relative Policy Optimization

GRPO (Shao et al., 2024) is one of the most effective and common methods to minimize $J(\pi_\theta)$. Each rollout includes LLM-generated tokens $a_1, ..., a_T$ (blue in Figure 2) and tool responses $s_1, ..., s_T$ (purple). Since tool responses are not generated by π_θ, they do not contribute to the policy gradient. The GRPO objective masks out tool-response tokens as follows:

$$J_{\text{single}}(\pi_\theta) \triangleq \mathbb{E}_{\substack{q\sim\mathcal{D} \\ \{\tau_i\}_{i=1}^G\sim\pi_{\theta_{\text{old}}}}} \left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{\sum_{t=1}^{T_i}|a_t^i|}\sum_{t=1}^{T_i} R_i^{\text{clip}}\right],$$
$$R_i^{\text{clip}} \triangleq \min(R_{i,t}(\theta)\hat{A}_{i,t}, \text{clip}(R_{i,t}(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,t}),$$
$$R_{i,t}(\theta) \triangleq \frac{\pi_\theta(a_t^i|[p_{\text{sys}}, q, a_1^i, s_1^i, ..., s_{t-1}^i])}{\pi_{\theta_{\text{old}}}(a_t^i|[p_{\text{sys}}, q, a_1^i, s_1^i, ..., s_{t-1}^i])},$$
$$\hat{A}_{i,t} \triangleq (r(\tau_i) - \text{mean}(\{r(\tau_i)\}_{i=1}^G))/\text{std}(\{r(\tau_i)\}_{i=1}^G),$$

where $\pi_{\theta_{\text{old}}}$ is a periodically updated snapshot of π_θ, G is the group size, $\tau_i \triangleq [a_1^i, s_1^i, \ldots, a_{T_i}^i]$ is the i-th rollout with T_i turns, $R_{i,t}(\theta)$ is the likelihood ratio between π_θ and $\pi_{\theta_{\text{old}}}$, $\hat{A}_{i,t}$ is the group-relative normalized reward, and $\text{clip}(\cdot, 1-\varepsilon, 1+\varepsilon)$ restricts values to $[1-\varepsilon, 1+\varepsilon]$.

3.3. Multi-Agent Multi-Turn Reinforcement Learning

As mentioned in the "introduction" section, multi-agent multi-turn TIP frameworks are designed to overcome the context length bottleneck and noisy tool-token issues present in single-agent multi-turn TIP. For clarity and without loss of generality, this paper considers a multi-agent framework with one planner-agent and one worker-agent. A multi-agent multi-turn TIP rollout is visualized in Figure 3. Specifically, q denotes the user query, and τ represents the entire multi-turn TIP rollout for handling it. p_{planner} is the system prompt specifying the role of the planner agent. At each turn t, the planner generates an action a_t containing a thinking block and either a subtask or the final answer, and receives a response s_t parsed from the worker agent's output. The planner proceeds for T turns in total. Each subtask query $q_{\text{sub\_}t}$ parsed from a_t is handled by a worker-agent rollout τ^t, guided by the system prompt p_{worker}. Within τ^t, the worker produces actions a_i^t (each including a thinking block and either a tool call or a final sub-answer) and receives tool responses s_i^t. Finally, r denotes the accuracy reward for the final planner answer a_T.

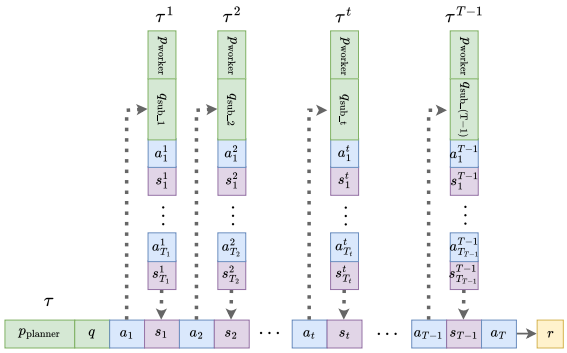


Figure 3. Visualization of a multi-agent multi-turn TIP rollout. The planner assigns subtasks to the worker, which completes them via multi-turn single-agent TIP iterations.

Each multi-agent rollout consists of T single-agent rollouts: one from the planner and $(T-1)$ from workers. Specifically:

$$\tau \triangleq [a_1, \tau^1, s_1, ..., a_{T-1}, \tau^{T-1}, s_{T-1}, a_T] \sim (\pi_\theta, \text{Tool}),$$
$$\tau^t \triangleq [a_1^t, s_1^t, ..., s_{T_t-1}^t, a_{T_t}^t],\ s_t \sim \text{Parse}(a_{T_t}^t), s_i^t \sim \text{Tool}(a_i^t),$$

where $\text{Parse}(a_{T_t}^t)$ is the worker-agent's response to the t-th subtask parsed from the final content in the worker-agent rollout, and $\text{Tool}(a_i^t)$ is the tool-response based on the parameters parsed from action a_i^t from the worker-agent.

Given a reward function $r(\cdot)$ that assigns 1 to correct answers and 0 to incorrect ones, the objective of multi-agent multi-turn RL can be formalized as:

$$\min_{\theta} J_{\text{multi}}(\pi_\theta) \triangleq \mathbb{E}_{q\sim\mathcal{D},\tau\sim(\pi_\theta,\text{Tool})}[r(\tau)],$$
$$a_t \sim \pi_\theta(\cdot\,|[p_{\text{planner}}, q, a_1, s_1, ..., s_{t-1}]),$$
$$a_j^t \sim \pi_\theta(\cdot\,|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, ..., s_{j-1}^t]),$$
$$s_t \sim \text{Parse}(a_{T_t}^t),\ q_{\text{sub\_}t} \sim \text{Parse}(a_t),\ s_j^t \sim \text{Tool}(a_j^t).$$

Notice that in $J_{\text{multi}}(\pi_\theta)$, a single LLM π_θ is deployed to serve as both the planner-agent and the worker-agent, distinguished only by different system prompts p_{planner} and p_{worker}. In this paper, we refer to this deployment configuration as **multi-agent-in-one-model**.

An alternative configuration is to deploy separate models for the planner-agent and worker-agents, which we refer to as multi-agent-multi-model. The multi-agent multi-turn RL objective can be directly generalized to this configuration. Let the planner-agent be parameterized by π_θ and K worker-agents be parameterized by $\pi_{\phi_1}, ..., \pi_{\phi_K}$. The resulting multi-agent-multi-model objective is

$$J_{\text{multi}}(\pi_\theta, \{\pi_{\phi_k}\}_{k\in[K]}) \triangleq \mathbb{E}_{q\sim\mathcal{D},\tau\sim(\pi_\theta,\{\pi_{\phi_k}\}_{k\in[K]},\text{Tool})}[r(\tau)],$$
$$a_t \sim \pi_\theta(\cdot\,|[p_{\text{planner}}, q, a_1, s_1, ..., s_{t-1}]),$$
$$a_j^t \sim \pi_{\phi_k}(\cdot\,|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, ..., s_{j-1}^t]),\ k \in [K],$$
$$s_t \sim \text{Parse}(a_{T_t}^t),\ (q_{\text{sub\_}t}, k) \sim \text{Parse}(a_t),\ s_j^t \sim \text{Tool}(a_j^t).$$

In this paper, we focus on exploring RL training under the multi-agent-in-one-model setting, as it offers several advantages over the multi-agent-multi-model setting: 1. The multi-agent-multi-model setting requires $(K+1)$ LLM rollout engines and additional RL infrastructure optimization. In contrast, the multi-agent-in-one-model framework uses only one single LLM rollout engine and remains compatible with off-the-shelf RL frameworks; 2. The multi-agent-in-one-model setting allows us to investigate whether RL training can benefit the model when it is exposed to experience from multiple agent roles.

4. Methodology

4.1. Multi-Agent Tool-Integrated Policy Optimization

A key challenge in the multi-agent-in-one-model setting is **credit assignment**: how should the planner-agent rollout τ^0 and the worker-agent rollouts τ^t share responsibility for the final accuracy of the full multi-turn TIP rollout τ? The planner-agent's final answer is directly verifiable, whereas worker-agent rollouts address unverifiable subtasks, making it essential to assess their contribution to the planner's final answer.

The core philosophy of MATPO is that, although the planner and worker have different roles, they exist within the *same rollout lifecycle*. Because their combined actions determine the success of a task, they are optimized against the same terminal reward, ensuring their policies are perfectly aligned.

We now derive the GRPO counterpart in the multi-agent-in-one-model setting to optimize $J_{\text{multi}}(\pi_\theta)$. The policy gradient $\nabla_\theta J_{\text{multi}}(\pi_\theta)$ equals

$$\begin{aligned}\nabla_\theta J_{\text{multi}}(\pi_\theta) &= \nabla_\theta \mathbb{E}_{q\sim\mathcal{D},\tau\sim(\pi_\theta,\text{Tool})}[r(\tau)] \\ &= \mathbb{E}_{q\sim\mathcal{D},\tau\sim(\pi_\theta,\text{Tool})}[r(\tau)\nabla_\theta \log \mathbb{P}_\theta(\tau)],\end{aligned}$$

where $r(\tau)$ is the accuracy reward for rollout τ and $\mathbb{P}_\theta(\tau)$ is the generation probability. Let $c_t^{\text{p}} \triangleq [p_{\text{planner}}, q, a_1, s_1, \ldots, s_{t-1}]$ and $c_j^{t,\text{w}} \triangleq [p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, \ldots, s_{j-1}^t]$ denote the planner and worker contexts, respectively. By the chain rule, the trajectory probability factorizes along planner–worker boundaries as

$$\mathbb{P}_\theta(\tau) = \prod_{t=1}^{T} \pi_\theta(a_t \mid c_t^{\text{p}}) \cdot \prod_{t=1}^{T-1} \mathbb{P}_\theta(\tau^t \mid a_t),$$
$$\mathbb{P}_\theta(\tau^t \mid a_t) = \prod_{j=1}^{T_t} \pi_\theta(a_j^t \mid c_j^{t,\text{w}}) \cdot \prod_{j=1}^{T_t-1} \mathbb{P}_{\text{Tool}}(s_j^t \mid a_j^t).$$

As the tool-responses are not generated by the LLM π_θ, it holds that $\nabla_\theta \mathbb{P}_{\text{Tool}}(s_j^t|a_j^t) = 0$, so taking the log-gradient yields

$$\begin{aligned}\nabla_\theta \log \mathbb{P}_\theta(\tau) = &\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t \mid c_t^{\text{p}}) \\ &+ \sum_{t=1}^{T-1}\sum_{j=1}^{T_t} \nabla_\theta \log \pi_\theta(a_j^t \mid c_j^{t,\text{w}}).\end{aligned}$$

A step-by-step derivation is provided in Appendix A.1. where $\tau^0 \triangleq [p_{\text{planner}}, q, a_1, s_1, .., s_{T-1}, a_T]$ is the planner rollout and τ^t is the t-th worker rollout.

Following standard GRPO derivation, the MATPO objective can be written as:

$$J_{\text{MATPO}}(\pi_\theta) \triangleq \mathbb{E}_{\substack{q\sim\mathcal{D} \\ \{\tau_i\}\sim(\pi_{\theta_{\text{old}}},\text{Tool})}} \left[\frac{1}{G}\sum_{i=1}^{G} \frac{1}{\sum_{t=0}^{T_i}|\tau_i^t|} \sum_{t=0}^{T_i} R_i^{\text{clip}}\right],$$
$$R_i^{\text{clip}} \triangleq \min(R_{i,t}\hat{A}_{i,t}, \text{clip}(R_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,t}),$$
$$\hat{A}_{i,t} \triangleq (r(\tau_i) - \text{mean}(\{r(\tau_i)\}_{i=1}^G))/\text{std}(\{r(\tau_i)\}_{i=1}^G),$$

where τ_i is the full rollout for query q with T_i subtasks, τ_i^0 is the planner rollout, τ_i^t $(t > 0)$ is the t-th worker rollout, and $\hat{A}_{i,t}$ is the group-relative normalized reward. $R_{i,t}$ is the likelihood ratio:

$$R_{i,t} \triangleq \begin{cases} \sum_{j=1}^{T_i} \dfrac{\pi_{\theta_{\text{old}}}(a_j^t|[p_{\text{planner}}, q, a_1, s_1, .., s_{j-1}])}{\pi_\theta(a_j^t|[p_{\text{planner}}, q, a_1, s_1, .., s_{j-1}])}, & t = 0, \\ \sum_{j=1}^{T_{i,t}} \dfrac{\pi_{\theta_{\text{old}}}(a_j^t|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, .., s_{j-1}^t])}{\pi_\theta(a_j^t|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, .., s_{j-1}^t])}, & t > 0, \end{cases}$$

where $T_{i,t}$ is the tool-calls count in the t-th subtask of τ_i.

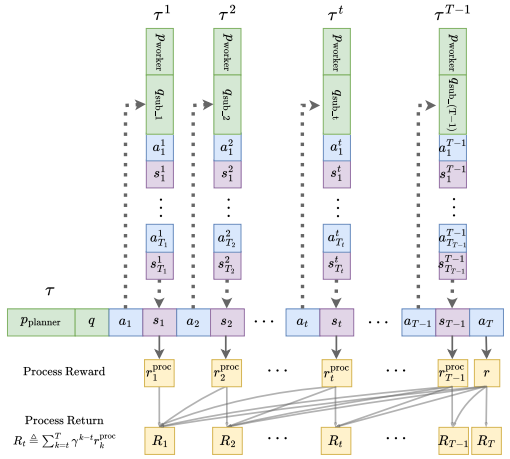


Figure 4. Visualization of MATPO-PR rollout. MATPO-PR assigns turn-level advantages by computing a discounted process return R_t from binary LLM-judged process rewards $\{r_i^{\text{proc}}\}_{i=t}^{T}$, enabling temporal credit assignment across planner–worker interactions instead of uniform rollout-level supervision.

We summarize the key distinctions between single-agent GRPO and MATPO as follows: unlike GRPO, which performs a single worker-agent rollout per update, MATPO executes one planner-agent rollout followed by T worker-agent rollouts. Moreover, while GRPO normalizes rewards across G worker rollouts for credit assignment, MATPO normalizes across $G \times (T+1)$ rollouts to jointly account for planner and worker contributions.

4.2. MATPO with Process Rewards

A limitation of standard MATPO is that it solely relies on sparse outcome-based rewards $r(\tau_i)$, which are assigned only upon the completion of the full rollout trajectory, leaving the extensive intermediate steps without direct supervision. This sparsity hinders the optimization of intermediate planner-worker interactions that are crucial for final success. To address this issue, we propose MATPO with Process Reward (**MATPO-PR**), which enables temporal credit assignment by attributing future success to specific intermediate planner-worker interactions. Specifically, upon receiving the worker response s_t at turn t, an LLM judge evaluates its information sufficiency for answering q. We assign a binary **process reward** $r_t^{\text{proc}} = 1$ if sufficient, and 0 otherwise. To facilitate temporal credit assignment, the **process return** $R_t \triangleq \sum_{k=t}^{T} \gamma^{k-t} r_k^{\text{proc}}$ is employed to quantify the value of the t-th planner-worker interaction, where R_t aggregates future process rewards via a discount factor γ. Intuitively, R_t captures the long-term value of the t-th interaction: a higher value implies that this step is beneficial in enabling subsequent high-quality actions and achieving a reliable final answer.

For each rollout group $\{\tau_i\}_{i=1}^{G}$ associated with each query q, we denote $R_{i,t}$ as the t-th process return of the i-th rollout. The MATPO-PR objective is modified from $J_{\text{MATPO}}(\pi_\theta)$ by replacing the outcome reward $r(\tau_i)$ with the process return $R_{i,t}$. Specifically, we replace the group-relative normalized reward $\hat{A}_{i,t}$ with $\hat{A}_{i,t}^{\text{proc}} \triangleq (R_{i,t} - \mu)/\sigma$, where μ and σ denote the mean and standard deviation of $\{R_{i,1}, ..., R_{i,T^i}\}_{i=1}^{G}$ over the group of all intermediate interactions within G rollouts, where T^i denotes the number of interactions in the i-th rollout τ_i.

Unlike standard MATPO, which assigns a uniform advantage to all tokens within a rollout, MATPO-PR enables fine-grained turn-level credit assignment, effectively incentivizing the proposal and accomplishment of subtasks that are instrumental to the final answer throughout the long-horizon trajectory.

4.3. Implementation Design

MATPO can be implemented on top of existing single-agent RL frameworks (*e.g.,* veRL). Figure 5 illustrates our implementation, which involves two key architectural designs.

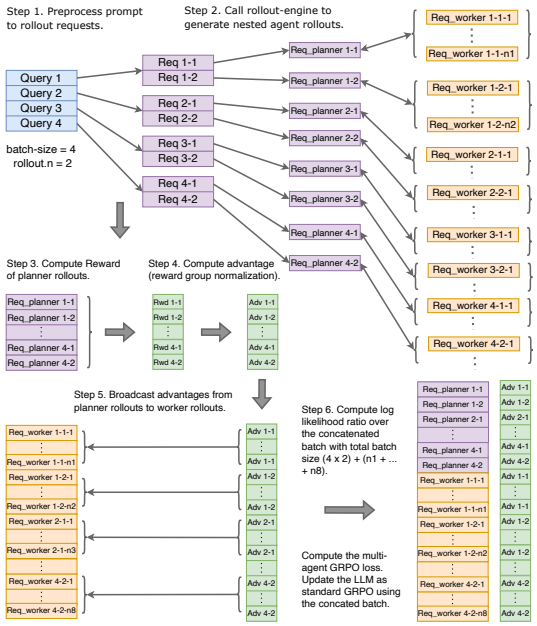


Figure 5. Implementation design of MATPO, showing nested rollouts and packed training batches.

Nested Rollout Mechanism. To reuse state-of-the-art rollout engine implementations, we implement multi-agent rollouts via a nested rollout mechanism: when a worker-agent is invoked within a planner rollout, a nested rollout function is launched inside the outer one, and these processes execute asynchronously. For each query, we feed `n.rollout` requests to the rollout engine (*e.g.,* `vLLM` and `SGLang`), generating `n.rollout` planner rollouts (purple boxes in

Figure 5), each with associated worker rollouts (orange boxes) for subtasks.

Training with Packed Rollouts. To ensure compatibility with data parallelism protocols, we pack worker-agent rollouts onto their corresponding planner-agent rollouts. Both are converted to data batches. We compute accuracy rewards by verifying final answers, normalize advantages across planner rollouts per query, and broadcast advantages to corresponding worker rollouts. Finally, planner and worker rollouts are concatenated into an augmented batch. We compute log likelihoods using π_θ and $\pi_{\theta_{\text{old}}}$, calculate $J_{\text{MATPO}}(\pi_\theta)$ while masking system prompts, queries, and tool responses, then update π_θ via standard optimization.

5. Experiments

5.1. Setups

In this work, we focus on the deep search scenario, where a planner-agent and a worker-agent comprise a two-agent system, aiming to find the answer to a given user query based on searching and web scraping [2]. Specifically, we implement our algorithm on top of veRL. The training hyperparameters are provided in the training script released in the GitHub repository. All experiments are conducted with 128 NVIDIA A800 80G GPUs.

Dataset and Base Model. All experiments are conducted on the `Qwen3-14B-base` model. We train the model with single-agent GRPO, MATPO , or MATPO-PR on a filtered subset of the `MuSiQue` (Trivedi et al., 2022) dataset, a multi-hop QA dataset. We remove overly difficult queries for which LLMs repeatedly fail to produce valid rollouts. Our models are then tested on `GAIA-text` (Mialon et al., 2024) [3], `WebWalkerQA` (Wu et al., 2025), and `FRAMES` (Krishna et al., 2025). All methods in our comparison share the same base model, training data, tool stack, and evaluator, so the comparison isolates the effect of the training algorithm.

Reward Function. In this work, we employ LLM-as-a-judge [4] to evaluate the accuracy of a model's answer against the ground-truth answer. The RL reward is set as `reward = 0.9 * acc + 0.1 * fmt`, where `acc` is a binary value indicating whether the rollout is correct, `fmt` measures the average correctness of the tool-calls generated by the model. Specifically, for single-agent RL, we define `fmt` as the success rate of all tool-call attempts parsed from the LLM's generated action. For MATPO, we define `fmt= 0.5 * fmt_p + 0.5 * fmt_w`, where `fmt_p` denotes the successful tool-call rate among a planner-agent rollout, and `fmt_w` denotes the average successful tool-call rate among all associated worker-agent rollouts. For MATPO-PR, the settings are similar.

Rollout Summary Mechanism. To encourage the agent to generate answers based on the entire rollout trajectory, we implement a final-summary mechanism. At the end of each rollout, we instruct the model to stop further tool calls and produce an answer based on a summary of the full rollout. We then perform an additional round of summarization and append this final summary to the complete rollout trajectory [5]. To avoid exceeding the model's maximum token length, if a rollout reaches the limit, we remove the latest messages from the trajectory until there is sufficient token budget for the final summary. Worker-agents in MATPO and MATPO-PR are equipped with this summary mechanism. Additional implementation details are provided in the Appendix, including MATPO-PR specifics (Appendix A.2.2) and agent system prompts with tool-call formats (Appendix A.2.1).

5.2. Results

MATPO consistently outperforms single-agent GRPO. Figure 6 presents the testing accuracy on `GAIA-text`, `WebWalkerQA`, and `FRAMES` across different training steps. MATPO consistently surpasses the single-agent GRPO baseline, underscoring the effectiveness of our approach. Specifically, MATPO achieves 36.89%, 30.29%, and 62.14% on `GAIA-text`, `WebWalkerQA`, and `FRAMES`, respectively, compared to 33.89%, 28.97%, and 57.34% for single-agent GRPO, leading to an average relative improvement of 7.26%. Moreover, MATPO exhibits more stable gains as training progresses. For instance, while the performance of single-agent GRPO drops after step 90 on both `GAIA-text` and `FRAMES`, MATPO continues to improve. We attribute this divergence to the vulnerability of single-agent training: agentic RL often suffers catastrophic drops in performance due to unstable environmental feedback (*e.g.,* missing or noisy responses from the Serper API). In contrast, MATPO can invoke additional browsing subtasks, enabling the agent to perform more robust searches and maintain steady progress.

MATPO-PR consistently outperforms MATPO. To highlight the stability introduced by incorporating process rewards, we evaluate multiple settings of γ, uniformly sampled from the interval $[0, 1]$, specifically $\{0.2, 0.5, 0.8\}$. A larger γ usually corresponds to larger process reward values. We first plot the accuracy reward curves on MuSiQue during training in Figure 7, where "MATPO-PR(γ)" is our result. As shown, "MATPO-PR" consistently achieves higher accuracy than both MATPO and the single-agent baseline.

[2] To avoid potential leakage of datasets hosted on HuggingFace, search results from this site are blocked by default, unless noted.

[3] `GAIA-text` is a curated subset of 103 text-only queries drawn from the `GAIA` dataset (Mialon et al., 2024), a benchmark for general AI assistants. Absolute scores on `GAIA-text` are therefore not directly comparable to numbers reported on the full `GAIA` benchmark, which additionally involves image, audio, and file-based tool use.

[4] We implement LLM-as-a-judge based on GPT-4o-mini with instructions shown in the Appendix.

[5] The rollout summary prompt is detailed in the Appendix.

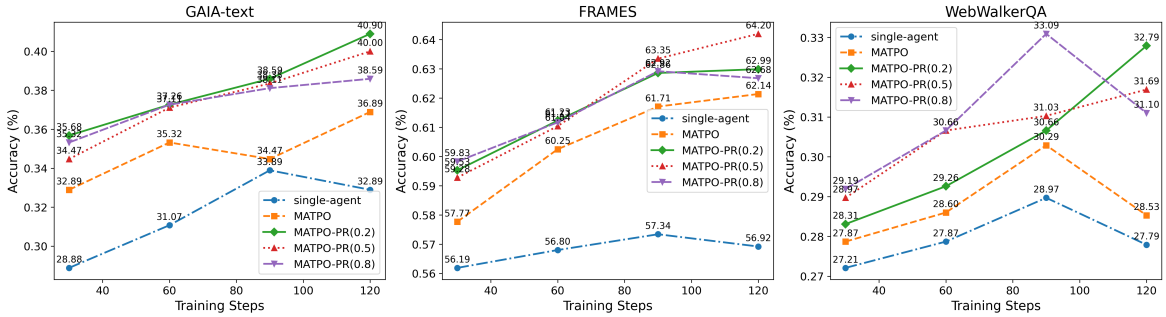


Figure 6. Test accuracy on three benchmarks across different training steps. Models are trained on the MusiQue dataset.

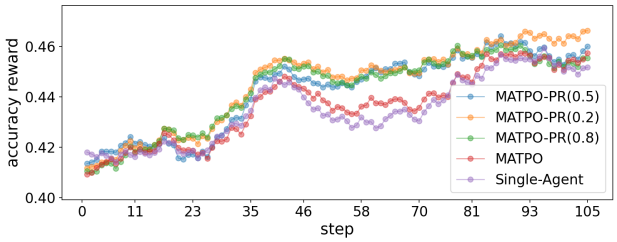


Figure 7. We record the evolution of training accuracy on the training set for different methods. All curves exhibit an upward trend, while MATPO-PR achieves the highest accuracy.

This advantage also persists on validation benchmarks, as shown in Figure 6. Our validation benchmarks include `GAIA-text`, `WebWalkerQA`, and `FRAMES`. From the validation accuracy curves, we observe that MATPO-PR remains almost entirely above the MATPO and single-agent curves, especially in the later stages of training. MATPO-PR achieves 40.90%, 33.09%, and 64.20% on `GAIA-text`, `WebWalkerQA`, and `FRAMES`, respectively, compared to 36.89%, 30.29%, and 62.14% for MATPO, leading to an average relative improvement of 7.81%.

We attribute the advantages of process rewards to two main factors. Firstly, we amplify the reward score (i.e., as reward return) at the step t where an accuracy reward $r_t^{\text{proc}} = 1$ first appears. This encourages the agent to reach high reward values more quickly. This effect can also be observed in Figure 8, where the visualization of process rewards for MATPO-PR exhibits high process reward values at an earlier stage. Secondly, we also strengthen the rewards after the first occurrence of $r_t^{\text{proc}} = 1$, preventing cases where a rollout achieves an accuracy of 1 after calling a certain sub-agent but then produces an incorrect answer in a subsequent step. This mechanism helps maintain a high and continuous proportion of high-reward steps within each rollout.

5.3. Visualization of Process Reward

The core motivation of MATPO-PR is to design a rational process reward, enhancing the execution quality of intermediate steps. To clearly illustrate this effect, we record the process rewards ($\{r_{i,1}^{\text{proc}}, ..., r_{i,T^i}^{\text{proc}}\}_{i=1}^{G}$ for each rollout group) of testing samples at different training steps. Although these reward sequences vary in length ($\{T^i\}_{i=1}^{G}$), they capture the underlying patterns of process reward dynamics. To enable a unified comparison across different rollouts and training steps, we interpolate each rollout's process reward sequence to a fixed length using nearest-neighbor interpolation, preserving original reward values.

Let the fixed sequence length be denoted as T, and the number of training steps as M. This process yields a $T \times M$ matrix, which provides a clear visualization of process reward evolution, as shown in Figure 8. From the figure, we observe that rollouts generated by our method achieve higher process rewards as training progresses, compared to the baseline MATPO. This trend is evidenced by the increased concentration of high-reward regions in the *upper-right portion of the matrix*. Moreover, as training progresses, rollouts produced by our method consistently exhibit a *higher proportion of steps with high process rewards* on testing benchmarks.

5.4. Ablation Studies and Practical Take-Aways

We conduct ablation studies on the key components of MATPO and MATPO-PR. Figure 9 shows the testing accuracy under different ablation settings. Each curve represents the following settings. (**Purple**, ▼): MATPO-PR(0.5); (**Brown**, +): MATPO-PR(0.5) + Diff.; (**Pink**, ×): MATPO-PR(0.5) + Partial; (**Orange**, ■): MATPO; (**Green**, ♦): MATPO without user query recapping; (**Red**, ▲): MATPO without final summary; (**Blue**, ●): single-agent GRPO. Higher curves reflect better accuracy.

Final summaries are necessary. Comparing **Orange** and **Red** curves in Figure 9, we find that adding a worker-agent summary significantly improves performance. Without a final summary, the planner-agent may be forced to consume the raw final block, which is error-prone: 1. Long worker-agent outputs may end with tool-call blocks instead of useful answers; 2. The `<think>...</think>` blocks from worker-agents can distract the planner-agent's subsequent actions. The final summary mitigates both issues, leading to a cleaner interface between the planner- and worker-agent.

Recapping the original user query to worker-agent improves the multi-agent RL performance. We find that the

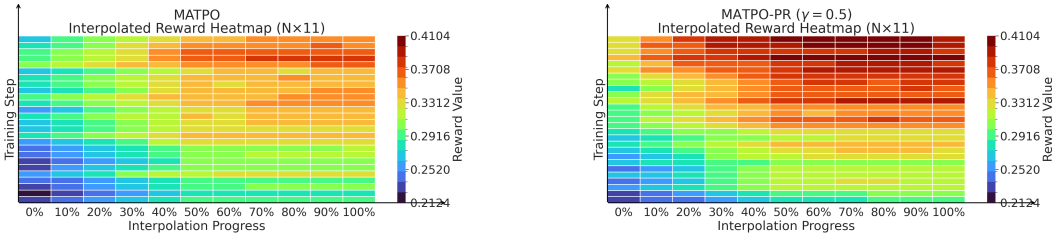


Figure 8. Visualization of process rewards ($\{r^{\text{proc}}_{i,1}, ..., r^{\text{proc}}_{i,T_i}\}_{i=1}^{G}$) on the `GAIA-text` test set, as training progresses.

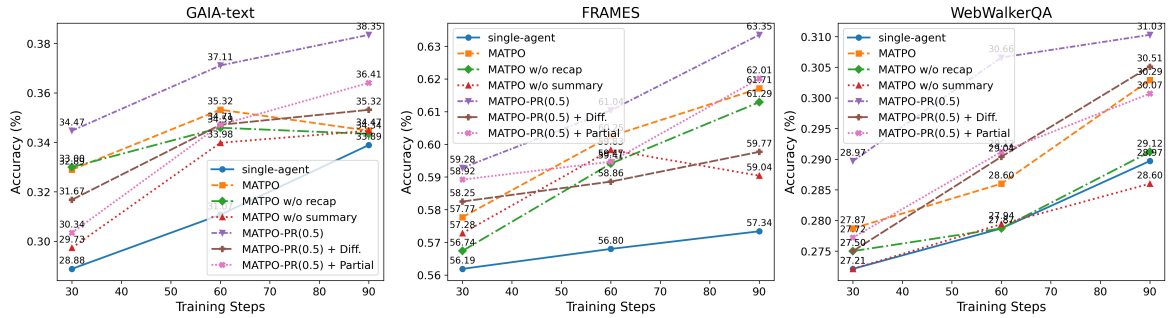


Figure 9. Test accuracy across different training steps for varying ablation settings. Models are trained on the MusiQue dataset.

context provided to the worker-agent (e.g., the input prompt) plays a crucial role in determining multi-agent RL performance. A comparison between the **Orange** and **Green** curves in Figure 9 illustrates this effect: recapping the original user query in the worker-agent's system prompt results in a substantial performance gain. We hypothesize that user query recapping provides the worker-agent explicit guidance toward fulfilling the original user query, thereby improving both the stability and quality of its browsing trajectory.

Cumulative rewards outperform differential rewards. We evaluate an alternative reward aggregation strategy, differential rewards, denoted as MATPO-PR+Diff. Specifically, suppose a rollout has a reward list $[r^{\text{proc}}_{i,1}, \ldots, r^{\text{proc}}_{i,T^i}]$, where r^{proc}_{i,T^i} is the final answer's reward. The gains of the process reward can be defined as $[r^{\text{proc}}_{i,1}, \ldots, r^{\text{proc}}_{i,T^j} - r^{\text{proc}}_{i,T^j-1}, \ldots, r^{\text{proc}}_{i,T^i} - r^{\text{proc}}_{i,T^i-1}]$, which can be viewed as the reward list instead. We then compute the final process returns $\{R_{i,t}\}_{t=1}^{T_i}$ using the "discounted sum of future process rewards" for this new reward list.

Comparing the **Brown** and **Purple** curves in Figure 9, we find that MATPO-PR+Diff leads to worse performance than MATPO-PR with cumulative rewards. The underlying reason is that this design effectively turns rewards into differences between consecutive sub-task executions. When a rollout executes a number of sub-tasks, the process rewards $\{r^{\text{proc}}_{i,t}\}_{t=1}^{T_i}$ may become similar for some consecutive steps, causing the computed differences to approach zero. As a result, the rewards of intermediate steps become sparse again, undermining the intended benefit.

Full-trajectory process returns are necessary. We evaluate another alternative process return strategy, partial-trajectory returns, denoted as MATPO-PR+Partial. This setting assumes that the process return for a rollout is primarily required for the steps preceding the first occurrence of an accuracy value of 1 (i.e., the first T^j where $r^{\text{proc}}_{i,T^j} = 1$). The rationale is that once the accuracy reaches 1 (indicating that the question can already be answered correctly), additional process return is no longer necessary for the following steps. Formally, suppose a rollout has a reward list $[r^{\text{proc}}_{i,1}, \ldots, r^{\text{proc}}_{i,T^i}]$, and let r^{proc}_{i,T^j} denote the first process reward that equals 1. In this case, the prefix $[r^{\text{proc}}_{i,1}, \ldots, r^{\text{proc}}_{i,T^j}]$ requires process return with higher priority. Therefore, we apply the computation of "discounted sum of future process" to $[r^{\text{proc}}_{i,1}, \ldots, r^{\text{proc}}_{i,T^j}]$ to obtain the process returns for steps $\{1, ..., T^j\}$, while keeping the remaining $[r^{\text{proc}}_{i,T^j+1}, \ldots, r^{\text{proc}}_{i,T^i}]$ unchanged as the process returns.

Comparing the **Pink** and **Purple** curves in Figure 9, we find that MATPO-PR+Partial performs worse than MATPO-PR with full-trajectory returns. This indicates that even after a process reward reaches 1 at a certain step, it can revert to 0 in subsequent steps. As a result, it is necessary to apply "discounted sum of future process" to the entire reward sequence $[r^{\text{proc}}_{i,1}, \ldots, r^{\text{proc}}_{i,T^i}]$ to achieve better performance.

6. Conclusions

This work introduces MATPO, a principled reinforcement learning framework for multi-agent-in-one-model training using role-specific prompts, along with MATPO-PR, an extension that addresses credit assignment via process rewards. Empirical evaluation across three deep research benchmarks demonstrates consistent improvements over single-agent baselines in both accuracy and robustness. More broadly, MATPO offers a new perspective on LLM post-training by

unifying multiple agent roles within a single model optimized against shared objectives. This paradigm enables richer learning signals from diverse tool interactions and role-specific experiences. A important future direction is to investigate scaling laws between role diversity and LLMs' agentic reasoning and planning capabilities.

Impact Statement

This paper presents work whose goal is to advance the field of post-training techniques for agentic foundation models, specifically in the area of reinforcement learning for multi-agent systems with tool integration. Our proposed methods, MATPO and MATPO-PR, aim to improve the capability of LLMs to coordinate multiple agent roles and effectively utilize external tools for complex reasoning tasks.

The potential positive societal impacts of this work include enabling more capable AI assistants for knowledge-intensive tasks such as research, information synthesis, and complex problem-solving. Improved multi-agent coordination could benefit applications in education, scientific discovery, and productivity tools.

As with advances in LLM capabilities more broadly, there are potential risks associated with more capable AI systems, including the possibility of generating misleading information or being misused for harmful purposes. However, our work focuses on improving coordination and tool use in controlled research settings, and we do not believe it introduces risks beyond those already present in the development of general-purpose language models.

We encourage the research community to continue developing responsible deployment practices and safety measures as multi-agent AI systems become more prevalent.

References

Chen, G., Zhang, Z., Cong, X., Guo, F., Wu, Y., Lin, Y., Feng, W., and Wang, Y. Learning evolving tools for large language models. In *Proceedings of ICLR*, 2025a.

Chen, Y., Wang, Y., Zhu, S., Yu, H., Feng, T., Zhang, M., Patwary, M., and You, J. Multi-agent evolve: Llm self-improve through co-evolution. *arXiv preprint arXiv:2510.23595*, 2025b.

Dong, G., Chen, Y., Li, X., Jin, J., Qian, H., Zhu, Y., Mao, H., Zhou, G., Dou, Z., and Wen, J.-R. Tool-star: Empowering llm-brained multi-tool reasoner via reinforcement learning. *arXiv preprint arXiv:2505.16410*, 2025.

Dong, G., Mao, H., Ma, K., Bao, L., Chen, Y., Wang, Z., Chen, Z., Du, J., Wang, H., Zhang, F., Zhou, G., Zhu, Y., Wen, J.-R., and Dou, Z. Agentic reinforced policy optimization. In *Proceedings of ICLR*, 2026.

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of ICML*, 2024.

Feng, J., Huang, S., Qu, X., Zhang, G., Qin, Y., Zhong, B., Jiang, C., Chi, J., and Zhong, W. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.

Hu, M., Zhou, Y., Fan, W., Nie, Y., Xia, B., Sun, T., Ye, Z., Jin, Z., Li, Y., Chen, Q., Zhang, Z., Wang, Y., Ye, Q., Ghanem, B., Luo, P., and Li, G. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation. In *Proceedings of NeurIPS*, 2025.

Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. In *Proceedings of COLM*, 2025.

Jin, H., Han, X., Yang, J., Jiang, Z., Liu, Z., Chang, C.-Y., Chen, H., and Hu, X. Llm maybe longlm: Selfextend llm context window without tuning. In *Proceedings of ICML*, 2024.

Khalifa, M., Agarwal, R., Logeswaran, L., Kim, J., Peng, H., Lee, M., Lee, H., and Wang, L. Process reward models that think. *arXiv preprint arXiv:2504.16828*, 2025.

Krishna, S., Krishna, K., Mohananey, A., Schwarcz, S., Stambler, A., Upadhyay, S., and Faruqui, M. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In *Proceedings of NAACL*, 2025.

Li, D., Jiang, B., Huang, L., Beigi, A., Zhao, C., Tan, Z., Bhattacharjee, A., Jiang, Y., Chen, C., Wu, T., Shu, K., Cheng, L., and Liu, H. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proceedings of EMNLP*, 2025a.

Li, K., Zhang, Z., Yin, H., Zhang, L., Ou, L., Wu, J., Yin, W., Li, B., Tao, Z., Wang, X., Shen, W., Zhang, J., Zhang, D., Wu, X., Jiang, Y., Yan, M., Xie, P., Huang, F., and Zhou, J. Websailor: Navigating super-human reasoning for web agent. *arXiv preprint arXiv:2507.02592*, 2025b.

Li, X. A review of prominent paradigms for llm-based agents: Tool use, planning (including rag), and feedback learning. In *Proceedings of COLING*, 2024.

Li, X., Zhao, R., Chia, Y. K., Ding, B., Joty, S., Poria, S., and Bing, L. Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources. In *Proceedings of ICLR*, 2024.

Li, X., Jin, J., Dong, G., Qian, H., Zhu, Y., Wu, Y., Wen, J.-R., and Dou, Z. Webthinker: Empowering large reasoning models with deep research capability. In *Proceedings of NeurIPS*, 2025c.

Li, X., Xu, W., Zhao, R., Jiao, F., Joty, S., and Bing, L. Can we further elicit reasoning in LLMs? critic-guided planning with retrieval-augmentation for solving challenging tasks. In *Proceedings of ACL*, 2025d.

Li, X., Zou, H., and Liu, P. Torl: Scaling tool-integrated rl. *arXiv preprint arXiv:2503.23383*, 2025e.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let's verify step by step. In *Proceedings of ICLR*, 2024.

Liu, B., Guertler, L., Yu, S., Liu, Z., Qi, P., Balcells, D., Liu, M., Tan, C., Shi, W., Lin, M., Lee, W. S., and Jaques, N. Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *arXiv preprint arXiv:2506.24119*, 2025a.

Liu, S., Liu, H., Liu, J., Xiao, L., Gao, S., Lyu, C., Gu, Y., Zhang, W., Wong, D. F., Zhang, S., and Chen, K. CompassVerifier: A unified and robust verifier for LLMs evaluation and outcome reward. In *Proceedings of EMNLP*, 2025b.

Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., and Scialom, T. Gaia: a benchmark for general ai assistants. In *Proceedings of ICLR*, 2024.

MiroMind. Mirorl: An mcp-first reinforcement learning framework for deep research agent. *GitHub*, 2025a.

MiroMind. Miroflow: An open-source agentic framework for deep research. *GitHub*, 2025b.

Motwani, S. R., Smith, C., Das, R. J., Rafailov, R., Torr, P., Laptev, I., Pizzati, F., Clark, R., and de Witt, C. S. MALT: Improving reasoning with multi-agent LLM training. In *Proceedings of COLM*, 2025.

Nguyen, X.-P., Pandit, S., Reddy, R. G., Xu, A., Savarese, S., Xiong, C., and Joty, S. Sfr-deepresearch: Towards effective reinforcement learning for autonomously reasoning single agents. *arXiv preprint arXiv:2509.06283*, 2025.

Noukhovitch, M., Huang, S., Xhonneux, S., Hosseini, A., Agarwal, R., and Courville, A. Faster, more efficient RLHF through off-policy asynchronous learning. In *Proceedings of ICLR*, 2025.

OpenAI. Deep research system card. *Technical Report*, 2025.

Ouyang, J., Yan, R., Luo, Y., Cheng, M., Liu, Q., Liu, Z., Yu, S., and Wang, D. Training powerful llm agents with end-to-end reinforcement learning, 2025.

Qian, C., Acikgoz, E. C., He, Q., Wang, H., Chen, X., Hakkani-Tür, D., Tur, G., and Ji, H. Toolrl: Reward is all tool learning needs. In *Proceedings of NeurIPS*, 2025.

Qin, S., Zhu, Y., Mu, L., Zhang, S., and Zhang, X. Metatool: Unleash open-world function calling capabilities of general-purpose large language models. In *Proceedings of ACL*, 2025.

Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., and Wen, J.-r. Tool learning with large language models: a survey. *Frontiers of Computer Science*, 2025.

Qu, Y., Zhang, T., Garg, N., and Kumar, A. Recursive introspection: Teaching LLM agents how to self-improve. In *Proceedings of ICML*, 2024.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In *Proceedings of NeurIPS*, 2023.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Singh, J., Pandya, Y., Vajreshwari, P., Magazine, R., and Nambi, A. Agentic reasoning and tool integration for LLMs via reinforcement learning. In *First Workshop on Foundations of Reasoning in Language Models*, 2025.

Tao, Z., Wu, J., Yin, W., Zhang, J., Li, B., Shen, H., Li, K., Zhang, L., Wang, X., Jiang, Y., Xie, P., Huang, F., and Zhou, J. Webshaper: Agentically data synthesizing via information-seeking formalization. *arXiv preprint arXiv:2507.15061*, 2025.

Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., and et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.

Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. Musique: Multihop questions via single-hop question composition. In *Proceedings of TACL*, 2022.

Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., Li, Y., Chen, D., Wu, Y., and Sui, Z. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of ACL*, 2024.

Wei, Z., Yao, W., Liu, Y., Zhang, W., Lu, Q., Qiu, L., Yu, C., Xu, P., Zhang, C., Yin, B., Yun, H., and Li, L. Webagent-r1: Training web agents via end-to-end multi-turn reinforcement learning. In *Proceedings of EMNLP*, 2025.

Wu, J., Yin, W., Jiang, Y., Wang, Z., Xi, Z., Fang, R., Zhang, L., He, Y., Zhou, D., Xie, P., and Huang, F. Webwalker: Benchmarking llms in web traversal. In *Proceedings of ACL*, 2025.

Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., and Sun, M. Infllm: training-free long-context extrapolation for llms with an efficient context memory. In *Proceedings of NeurIPS*, 2024.

Xu, R. and Peng, J. A comprehensive survey of deep research: Systems, methodologies, and applications. *arXiv preprint arXiv:2506.12594*, 2025.

Xue, Z., Zheng, L., Liu, Q., Li, Y., Zheng, X., Ma, Z., and An, B. Simpletir: End-to-end reinforcement learning for multi-turn tool-integrated reasoning. *arXiv preprint arXiv:2509.02479*, 2025.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., and et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In *Proceedings of ICLR*, 2023.

Zhang, J., Zhu, Y., Sun, M., Luo, Y., Qiao, S., Du, L., Zheng, D., Chen, H., and Zhang, N. Lightthinker: Thinking step-by-step compression. In *Proceedings of EMNLP*, 2025.

Zhao, R., Li, X., Joty, S., Qin, C., and Bing, L. Verify-and-edit: A knowledge-enhanced chain-of-thought framework. In *Proceedings of ACL*, 2023.

Zhong, H., Shan, Z., Feng, G., Xiong, W., Cheng, X., Zhao, L., He, D., Bian, J., and Wang, L. DPO meets PPO: Reinforced token optimization for RLHF. In *Proceedings of ICML*, 2025.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. Webarena: A realistic web environment for building autonomous agents. In *Proceedings of ICLR*, 2024a.

Zhou, Y., Zanette, A., Pan, J., Levine, S., and Kumar, A. ArCHer: Training language model agents via hierarchical multi-turn RL. In *Proceedings of ICML*, 2024b.

Zhou, Z., Tao, R., Zhu, J., Luo, Y., Wang, Z., and Han, B. Can language models perform robust reasoning in chain-of-thought prompting with noisy rationales? In *Proceedings of NeurIPS*, 2024c.

A. Appendix

A.1. Step-by-Step Derivation of the MATPO Policy Gradient

This appendix gives the full derivation behind the compact form presented in the main text. Starting from

$$\begin{aligned}\nabla_\theta J_{\text{multi}}(\pi_\theta) &= \nabla_\theta \mathbb{E}_{q\sim\mathcal{D},\tau\sim(\pi_\theta,\text{Tool})}[r(\tau)] \\ &= \mathbb{E}_{q\sim\mathcal{D},\tau\sim(\pi_\theta,\text{Tool})}[r(\tau)\nabla_\theta \log \mathbb{P}_\theta(\tau)],\end{aligned}$$

we expand the trajectory probability along the planner–worker sequence. The planner emits T actions $a_1, \ldots, a_T$ interleaved with $T-1$ worker sub-rollouts $\tau^1, \ldots, \tau^{T-1}$ and their parsed responses $s_1, \ldots, s_{T-1}$, giving

$$\begin{aligned}\mathbb{P}_\theta(\tau) &\triangleq \mathbb{P}_\theta([p_{\text{planner}}, q, a_1, \tau^1, s_1, \ldots, \tau^{T-1}, s_{T-1}, a_T]) \\ &= \pi_\theta(a_1|[p_{\text{planner}}, q])\, \mathbb{P}_\theta(\tau^1|a_1) \cdots \mathbb{P}_\theta(\tau^{T-1}|a_{T-1}) \\ &\quad \cdot \pi_\theta(a_T|[p_{\text{planner}}, q, a_1, \ldots, s_{T-1}]).\end{aligned}$$

Each worker sub-rollout $\tau^t = [a_1^t, s_1^t, \ldots, s_{T_t-1}^t, a_{T_t}^t]$ contains T_t worker actions interleaved with $T_t - 1$ tool responses, giving

$$\begin{aligned}\mathbb{P}_\theta(\tau^t|a_t) &\triangleq \mathbb{P}_\theta([p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, \ldots, s_{T_t-1}^t, a_{T_t}^t]) \\ &= \pi_\theta(a_1^t|[p_{\text{worker}}, q_{\text{sub\_}t}])\, \mathbb{P}_{\text{Tool}}(s_1^t|a_1^t) \\ &\quad \cdot \pi_\theta(a_2^t|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t]) \cdots \mathbb{P}_{\text{Tool}}(s_{T_t-1}^t|a_{T_t-1}^t) \\ &\quad \cdot \pi_\theta(a_{T_t}^t|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, \ldots, s_{T_t-1}^t]).\end{aligned}$$

Since tool responses are not generated by π_θ, we have $\nabla_\theta \log \mathbb{P}_{\text{Tool}}(s_j^t|a_j^t) = 0$. Taking the log-gradient of $\mathbb{P}_\theta(\tau)$ then expanding each factor yields

$$\begin{aligned}\nabla_\theta \log \mathbb{P}_\theta(\tau) =& \nabla_\theta \Big(\log \pi_\theta(a_1|[p_{\text{planner}}, q]) + \log \mathbb{P}_\theta(\tau^1|a_1) + \cdots \\ &+ \log \mathbb{P}_\theta(\tau^{T-1}|a_{T-1}) + \log \pi_\theta(a_T|[p_{\text{planner}}, q, a_1, \ldots, s_{T-1}]) \Big) \\ =& \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t|[p_{\text{planner}}, q, a_1, s_1, \ldots, s_{t-1}]) \\ &+ \sum_{t=1}^{T-1} \sum_{j=1}^{T_t} \nabla_\theta \log \pi_\theta(a_j^t|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, \ldots, s_{j-1}^t]) \\ =& \sum_{t=1}^{T} \frac{\nabla_\theta \pi_\theta(a_t|[p_{\text{planner}}, q, a_1, s_1, \ldots, s_{t-1}])}{\pi_\theta(a_t|[p_{\text{planner}}, q, a_1, s_1, \ldots, s_{t-1}])} \\ &+ \sum_{t=1}^{T-1} \sum_{j=1}^{T_t} \frac{\nabla_\theta \pi_\theta(a_j^t|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, \ldots, s_{j-1}^t])}{\pi_\theta(a_j^t|[p_{\text{worker}}, q_{\text{sub\_}t}, a_1^t, s_1^t, \ldots, s_{j-1}^t])},\end{aligned}$$

matching the compact form given in the main text. Substituting back into $\nabla_\theta J_{\text{multi}}$ and applying standard GRPO clipping with group-relative normalization yields the J_{MATPO} objective.

A.2. Implementation Details

A.2.1. Agent System Prompt and Tool-Call Format

We adopt an XML format to parse tool calls from both planner and worker agents. The planner-agent's system prompt specifies the tool schema for invoking the worker-agent, while the worker-agent's system prompt defines the schema

for Google's Serper API (used for search and scraping). After each tool call, the tool's response is wrapped as a "user message" and appended to the agent's rollout trajectory. To help the worker agent execute the user's original query from the planner agent, we include a recap of the query in the worker agent's system prompt, a technique we refer to as *user query recapping*. The detailed system prompts and tool schemas are provided in Sec. A.4.

A.2.2. Implementation Details for MATPO-PR

The process reward is computed by requiring the main agent to produce a summary based on the information available up to a specific step, i.e., immediately after completing a sub-task. After the process reward is obtained, the summary operation and its generated content are removed from the main agent's trajectory. As a result, this auxiliary summarization step does not affect the agent's trajectory. This design preserves the original workflow of MATPO while augmenting it with additional process-level supervision.

We place the summary operation after all sub-agents complete their tasks, as the main agent may invoke multiple sub-agents to accomplish different sub-tasks within a single step. In this case, the summary operation is executed only after all sub-agents have finished their respective sub-tasks. Consequently, a single, well-defined reward value is assigned at this step to serve as the process reward.

Similar to MATPO, the format reward for the main agent is defined as the average of the format reward computed from the main agent's rollout and the mean format reward across all sub-agents' rollouts, as `fmt= 0.5 * fmt_p + 0.5 * fmt_w`. Conversely, the format reward for each sub-agent is defined as the average of the format reward from its own rollout (`fmt_wi`) and the format reward from the main agent's rollout (`fmt_p`), as `fmt= 0.5 * fmt_wi + 0.5 * fmt_p`.

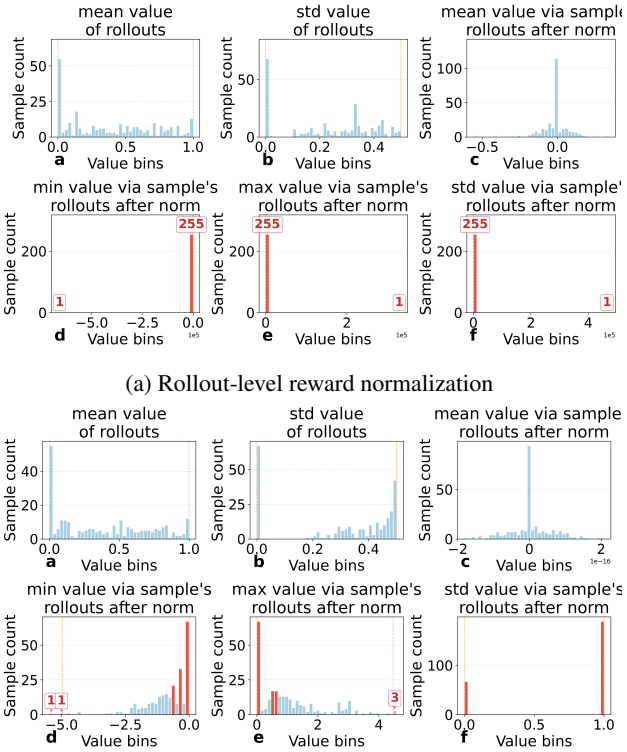
We first allocate a dedicated storage to record the intermediate summaries generated after invoking the sub-agents. These summaries are then evaluated by an LLM-as-judge to produce corresponding scores, which are used to derive the process rewards. During the evaluation stage, each summary is sent to the LLM independently for scoring.

A.3. Analysis of MATPO-PR

A.3.1. Analysis of the Normalization Strategy

When performing multi-agent GRPO, it is necessary to normalize the rewards in the rollout group $\{\tau_i\}_{i=1}^{G}$ to stabilize RL training. However, unlike the previous MATPO setting, the introduction of process rewards means that each rollout now contains multiple reward values. The normalization strategy must explicitly account for all process rewards.

We consider two possible approaches to address this. 1. During training, suppose that for each task there are G rollouts (as $\{\tau_i\}_{i=1}^{G}$), and the i-th rollout contains $T^i - 1$ sub-tasks. In this case, the reward sequence of the i-th rollout has length T^i. We take the average of these T^i reward values as the "rollout-level reward" for the i-th rollout ($\{r_{i,1}^{\text{proc}}, ..., r_{i,T^i}^{\text{proc}}\}$), obtaining μ_i. Then we compute the mean and standard deviation (μ and σ) over $\{\mu_i\}_{i=1}^{G}$ to perform normalization in GRPO. 2. We first collect the reward lists from all G rollouts, as $\{r_{1,1}^{\text{proc}}, ..., r_{1,T^1}^{\text{proc}}, ..., r_{G,1}^{\text{proc}}, ..., r_{G,T^i}^{\text{proc}}\}$. We then compute the mean and standard deviation over all reward values in this list jointly, and use them to perform normalization in GRPO. The former uses **rollout-level** normalization parameters, whereas the latter adopts more fine-grained, **step-level** normalization parameters.



(a) Rollout-level reward normalization

(b) Step-level reward normalization

Figure 10. We record the mean and standard deviation of rollout rewards ($\{r_{i,1}^{\text{proc}}, ..., r_{i,T^i}^{\text{proc}}\}_{i=1}^{G}$) before normalization (a and b), and compute the mean, maximum, minimum, and standard deviation after normalization (c, d, e, and f). As shown, using *rollout-level* rewards to estimate normalization parameters can produce extreme normalized values that destabilize training, whereas *step-level* normalization yields more stable and well-behaved reward distributions. These statistics are reported for MuSiQue under the "MATPO-PR ($\gamma = 0.5$)" setting at training step 60.

However, we demonstrate that the first approach is problematic. With the introduction of process rewards, rollouts within a group can have different reward sequences while sharing the same mean value. In such cases, the standard

Pattern Type	Previous released MATPO rollouts	Reproduced rollouts using released MATPO model	Rollouts of new trained MATPO
peopleAlsoAsk	400/1786=22.39%	2251/2811=80.08%	1312/1731=75.79%
relatedSearches	805/1786=45.07%	2382/2811=84.74%	1344/1731=77.64%
answerBox	169/1786=9.46%	896/2811=31.87%	540/1731=31.20%

Table 1. Statistics of the Serper parameters used in Google Search: the proportion of trajectories containing the corresponding returned parameters in Google Search–based executions.

deviation becomes **zero**, leading to extreme values in the normalized rewards, as illustrated in Figure 10. The second normalization method avoids this issue by directly accounting for individual process reward values across all rollouts. For sub-agents, we apply the normalization parameters derived from the main agent.

A.3.2. Note on Result Variability Due to External Tools

During our experiments, we observed that agent performance (e.g., MATPO) may vary over time due to changes in the external tools and servers they access.

Upon investigation, we identified the primary cause as changes in the Serper API, a critical tool for Google search and web scraping. Specifically, we found that this API has recently undergone parameter changes, particularly in its Google search component.

Specifically, we analyzed the Google search parameters used in the previous MATPO rollouts (run several months ago) and compared them with those used in our current experiments (a period shortly before submission), focusing on three parameters: *peopleAlsoAsk*, *relatedSearches*, and *answerBox*. We found that the proportions of these parameters differed substantially between the two settings. In the current experiments, these parameters are returned at a much higher rate, whereas they appeared only rarely in the previous MATPO runs. As a result, the Google search calls in the current Serper configuration return significantly more information, which may introduce noisy or distracting content and substantially increase input length. Both factors can potentially degrade accuracy. Detailed statistics of these parameters are reported in Table 1.

A.4. Prompts

A.4.1. System Prompt and Tool Schema of the Planner-Agent

System Prompt:

```
In this environment you have access to a set of tools you can use to answer the user's question.

You only have access to the tools provided below. You can only use one tool per message, and will receive the result of that tool in the user's next response. You use tools step-by-step to accomplish a given task, with each tool-use informed by the result of the previous tool-use. Today is: 2025-07-16

# Tool-Use Formatting Instructions

Tool-use is formatted using XML-style tags. The tool-use is enclosed in <use_mcp_tool></use_mcp_tool> and each parameter is similarly enclosed within its own set of tags.

The Model Context Protocol (MCP) connects to servers that provide additional tools and resources to extend your capabilities. You can use the server's tools via the `use_mcp_tool`.

Description:
Request to use a tool provided by a MCP server. Each MCP server can provide multiple tools with different capabilities. Tools have defined input schemas that specify required and optional parameters.

Parameters:
- server_name: (required) The name of the MCP server providing the tool
- tool_name: (required) The name of the tool to execute
- arguments: (required) A JSON object containing the tool's input parameters, following the tool's input schema, quotes within string must be properly escaped, ensure it's valid JSON

Usage:
<use_mcp_tool>
<server_name>server name here</server_name>
<tool_name>tool name here</tool_name>
<arguments>
{
 "param1": "value1",
```

```
 "param2": "value2 "escaped string
     ""
}
</arguments>
</use_mcp_tool>

Important Notes:
- Tool-use must be placed **at the end** of your response, **top-level**, and not nested within other tags.
- Always adhere to this format for the tool use to ensure proper parsing and execution.

String and scalar parameters should be specified as is, while lists and objects should use JSON format. Note that spaces for string values are not stripped. The output is not expected to be valid XML and is parsed with regular expressions.



Here are the functions available in JSONSchema format:

## Server name: browsing_agent
### Tool name: search_and_browse
Description: This tool is an agent that performs the subtask of searching and browsing the web for specific missing information and generating the desired answer. The subtask should be clearly defined, include relevant background, and focus on factual gaps. It does not perform vague or speculative subtasks.
Args:
  subtask: the subtask to be performed.
Returns:
  the result of the subtask.
Input JSON schema: {'properties': {'subtask': {'title': 'Subtask', 'type': 'string'}}, 'required': ['subtask'], 'title': 'search_and_browseArguments', 'type': 'object'}



# General Objective

You accomplish a given task iteratively, breaking it down into clear steps and working through them methodically.


## Task Strategy

1. Analyze the user's request and set clear, achievable sub-goals. Prioritize these sub-goals in a logical order.
2. Start with a concise, numbered, step-by-step plan outlining how you will solve the task before taking any action.
3. Work through these sub-goals sequentially. After each step, adjust your plan as needed.
4. Use tools strategically to accomplish each sub-goal.
5. Revise earlier steps if new information emerges.

## Tool-Use Guidelines

1. Each step must involve a single tool call, unless the task is already solved.
2. Before each tool call:
   - Summarize what is known.
   - Identify what is missing.
   - Choose the most relevant tool.
   - Verify all required parameters.
3. All tool queries must include full context.
4. Avoid vague queries. Each call should retrieve actionable information.
5. Extract and summarize partial information if a tool result is incomplete.

## Tool-Use Communication Rules

1. Do not include tool results in your response.
2. Do not present the final answer until the entire task is complete.
3. Do not mention tool names.
4. Do not engage in unnecessary back-and-forth.
5. Do not use non-existent tools.
6. Respond in the same language as the user's message.
7. If the task does not require tool use, answer directly.


# Agent Specific Objective

You are a task-solving agent that uses tools step-by-step to answer the user's question. Your goal is to provide
```

```
        complete, accurate and well-
        reasoned answers using
        additional tools.
```

A.4.2. System Prompt and Tool Schema of the Worker-Agent

System Prompt:

```
In this environment you have access to a set of tools you can use to answer the user's question.

You only have access to the tools provided below. You can only use one tool per message, and will receive the result of that tool in the user's next response. You use tools step-by-step to accomplish a given task, with each tool-use informed by the result of the previous tool-use. Today is: 2025-07-08

# Tool-Use Formatting Instructions

Tool-use is formatted using XML-style tags. The tool-use is enclosed in <use_mcp_tool></use_mcp_tool> and each parameter is similarly enclosed within its own set of tags.

The Model Context Protocol (MCP) connects to servers that provide additional tools and resources to extend your capabilities. You can use the server's tools via the `use_mcp_tool`.

Description:
Request to use a tool provided by a MCP server. Each MCP server can provide multiple tools with different capabilities. Tools have defined input schemas that specify required and optional parameters.

Parameters:
- server_name: (required) The name of the MCP server providing the tool
- tool_name: (required) The name of the tool to execute
- arguments: (required) A JSON object containing the tool's input parameters, following the tool's input schema, quotes within string must be properly escaped, ensure it's valid JSON

Usage:
<use_mcp_tool>
<server_name>server name here</server_name>
<tool_name>tool name here</tool_name>
<arguments>
{
 "param1": "value1",
 "param2": "value2 \"escaped string\""
}
</arguments>
</use_mcp_tool>

Important Notes:
- Tool-use must be placed **at the end** of your response, **top-level**, and not nested within other tags.
- Always adhere to this format for the tool use to ensure proper parsing and execution.

String and scalar parameters should be specified as is, while lists and objects should use JSON format. Note that spaces for string values are not stripped. The output is not expected to be valid XML and is parsed with regular expressions.


Here are the functions available in JSONSchema format:

## Server name: search_and_scrape_webpage
### Tool name: google_search
Description: Tool to perform web searches via Serper API and retrieve rich results. It is able to retrieve organic search results, people also ask, related searches, and knowledge graph.
Input JSON schema: {'type': 'object', 'properties': {'q': {'type': 'string', 'description': 'Search query string'}, 'gl': {'type': 'string', 'description': "Optional region code for search results in ISO 3166-1 alpha-2 format (e.g., 'us')"}, 'hl': {'type': 'string', '
```

```
        description': "Optional
        language code for search
        results in ISO 639-1 format (e
        .g., 'en')"}, 'location': {'
        type': 'string', 'description
        ': "Optional location for
        search results (e.g., 'SoHo,
        New York, United States', '
        California, United States')"},
         'num': {'type': 'number', '
        description': 'Number of
        results to return (default:
        10)'}, 'tbs': {'type': 'string
        ', 'description': "Time-based
        search filter ('qdr:h' for
        past hour, 'qdr:d' for past
        day, 'qdr:w' for past week, '
        qdr:m' for past month, 'qdr:y'
         for past year)"}, 'page': {'
        type': 'number', 'description
        ': 'Page number of results to
        return (default: 1)'}, '
        autocorrect': {'type': '
        boolean', 'description': '
        Whether to autocorrect
        spelling in query'}}, '
        required': ['q', 'gl', 'hl']}
44
45  ### Tool name: scrape
46  Description: Tool to scrape a
        webpage and retrieve the text
        and, optionally, the markdown
        content. It will retrieve also
         the JSON-LD metadata and the
        head metadata.
47  Input JSON schema: {'type': '
        object', 'properties': {'url':
         {'type': 'string', '
        description': 'The URL of the
        webpage to scrape.'}, '
        includeMarkdown': {'type': '
        boolean', 'description': '
        Whether to include markdown
        content.', 'default': False}},
         'required': ['url']}
48
49
50
51  # General Objective
52
53  You accomplish a given task
        iteratively, breaking it down
        into clear steps and working
        through them methodically.
54
55  ## Task Strategy
56
57  1. Analyze the user's request and
        set clear, achievable sub-
        goals. Prioritize these sub-
        goals in a logical order.
58  2. Start with a concise, numbered,
         step-by-step plan outlining
        how you will solve the task
        before taking any action.
59  3. Work through these sub-goals
        sequentially. After each step,
         adjust your plan as needed.
60  4. Use tools strategically to
        accomplish each sub-goal.
61  5. Revise earlier steps if new
        information emerges.
62
63  ## Tool-Use Guidelines
64
65  1. Each step must involve a single
         tool call, unless the task is
         already solved.
66  2. Before each tool call:
67     - Summarize what is known.
68     - Identify what is missing.
69     - Choose the most relevant tool
           .
70     - Verify all required
           parameters.
71  3. All tool queries must include
        full context.
72  4. Avoid vague queries. Each call
        should retrieve actionable
        information.
73  5. Extract and summarize partial
        information if a tool result
        is incomplete.
74
75  ## Tool-Use Communication Rules
76
77  1. Do not include tool results in
        your response.
78  2. Do not present the final answer
         until the entire task is
        complete.
79  3. Do not mention tool names.
80  4. Do not engage in unnecessary
        back-and-forth.
81  5. Do not use non-existent tools.
82  6. Respond in the same language as
         the user's message.
83  7. If the task does not require
        tool use, answer directly.
84
85  # Agent Specific Objective
86
87  You are a task-solving agent that
        uses tools step-by-step to
        answer the user's question.
        Your goal is to provide
        complete, accurate and well-
        reasoned answers using
        additional tools.
```

A.5. Instruction Prompt for Rollout Summarization

System Prompt:

```
[SYSTEM]
This is a direct instruction to you. This is your final turn. You MUST NOT use any tools.
Your task is to provide a final, structured report summarizing all the information you have gathered to answer your assigned subtask.

[CONTEXT]
The main task was: "{main_query}"
Your assigned subtask was: "{task_description}"
Your assigned subtask was intended to help solve the main task.

[INSTRUCTIONS]

{failed_instruction}

Your final response MUST be a clear, complete, and structured report in markdown format.
Organize the content into logical sections with the following headings: `## Conclusion`, `## Supporting Information`, `## Observations`, and `## Contribution to Main Task`.

- **CRITICAL**: Do NOT include raw URLs. Replace any URLs with `([link])`.
- Your response should only contain factual, specific, and well-organized information based on your previous actions.
- Do not include speculative filler, vague summaries, or conversational text.

Here is an example of the required format:

# Final Response: [Title summarizing the subtask]

## Conclusion:
[A concise summary of your findings and the final answer for the subtask. Bold key information.]

## Supporting Information:
[Detailed supporting facts, data, or quotes you discovered. Use bullet points or numbered lists for clarity.]
- Source 1: Brief description of finding 1.
- Source 2: Brief description of finding 2.

## Observations:
[Any additional context, confidence level, or notes on how the conclusion was reached.]

## Contribution to Main Task:
[Explain how the answer to your subtask helps solve the overall main task. What are the next steps the main agent should consider?]
```

A.6. Instruction Prompt for Process Summarization to Compute Process Rewards

System Prompt:

```
[SYSTEM]
This is a direct instruction to you. This is your final turn. You MUST NOT use any tools.
Your task is to provide a final, structured report summarizing the entire conversation and providing a definitive answer to the original task.

The original task was: "{main_query}"

[INSTRUCTIONS]

{failed_instruction}

Your final response MUST be a clear, complete, and structured report in markdown format.
Organize the content into logical sections with the following headings: `## Conclusion`, `## Supporting Information`, and `## Observations`.

- **CRITICAL**: Do NOT include any raw URLs.
- Your response should only contain factual, specific, and well-organized information based on the conversation.
- Do not include speculative filler, vague summaries, or conversational text.
- You should wrap the final answer
```

```
        to the original task in \\
       boxed{{}}, in `## Conclusion`.

Here is an example of the required
     format:

# Final Response: [Title
    summarizing the task]

## Conclusion:
[A concise summary of your
    findings and the final answer
    to the user's question. Bold
    key information. If the answer
     is a number or short phrase,
    provide it directly. You
    should wrap the final answer
    to the original task in \\
    boxed{{}}]

## Supporting Information:
[Detailed supporting facts, data,
    or quotes from the
    conversation that support your
     conclusion. Use bullet points
     or numbered lists for clarity
    .]
- Source 1: Brief description of
    finding 1.
- Source 2: Brief description of
    finding 2.

## Observations:
[Any additional context,
    confidence level, or notes on
    how the final answer was
    derived from the conversation
    history.]
```

A.7. Instruction Prompt for LLM-as-Judge.

System Prompt:

```
Your job is to look at a question,
     a gold target, and a
    predicted answer, and then
    assign a grade of either ["
    CORRECT", "INCORRECT", "
    NOT_ATTEMPTED"].
First, I will give examples of
    each grade, and then you will
    grade a new example.


The following are examples of
    CORRECT predicted answers.
```
```
Question: What are the names of
    Barack Obama's children?
Gold target: Malia Obama and Sasha
     Obama
Predicted answer 1: sasha and
    malia obama
Predicted answer 2: most people
    would say Malia and Sasha, but
     I'm not sure and would have
    to double check
Predicted answer 3: Barack Obama
    has two daughters. Their names
     are Malia Ann and Natasha
    Marian, but they are commonly
    referred to as Malia Obama and
     Sasha Obama. Malia was born
    on July 4, 1998, and Sasha was
     born on June 10, 2001.
```
```
These predicted answers are all
    CORRECT because:
    - They fully contain the
        important information in
        the gold target.
    - They do not contain any
        information that
        contradicts the gold
        target.
    - Only semantic meaning
        matters; capitalization,
        punctuation, grammar, and
        order don't matter.
    - Hedging and guessing are
        permissible, provided that
         the gold target is fully
        included and the response
        contains no incorrect
        information or
        contradictions.


The following are examples of
    INCORRECT predicted answers.
```
```
Question: What are the names of
    Barack Obama's children?
Gold target: Malia and Sasha
Predicted answer 1: Malia.
Predicted answer 2: Malia, Sasha,
    and Susan.
Predicted answer 3: Barack Obama
    does not have any children.
Predicted answer 4: I think it's
    either Malia and Sasha. Or it
    could be Malia and Jackie. Or
    it could be Joey and Malia.
Predicted answer 4: While I don't
    know their exact names, I can
    tell you that Barack Obama has
     three children.
Predicted answer 5: It's possible
    you may mean Betsy and Olivia.
     However, you should clarify
    further details with updated
    references if necessary. Is
    that the correct answer?
Predicted answer 6: It may be the
```

```
        case that Obama's child is
        named James. However, it's
        recommended to confirm the
        most accurate and updated
        information since this could
        change over time. This model
        may not always reflect the
        most current information.
31  ```
32  These predicted answers are all
        INCORRECT because:
33      - A factual statement in the
            answer contradicts the
            gold target. Incorrect
            statements that have some
            hedging (e.g., "it is
            possible that", "although
            i'm not sure, i think")
            are also considered
            incorrect.
34
35
36  The following are examples of
        NOT_ATTEMPTED predicted
        answers.
37  ```
38  Question: What are the names of
        Barack Obama's children?
39  Gold target: Malia and Sasha
40  Predicted answer 1: I don't know.
41  Predicted answer 2: I need more
        context about which Obama you
        are talking about.
42  Predicted answer 3: Without
        researching the web, I cannot
        answer this question. However,
         I can tell you that Barack
        Obama has two children.
43  Predicted answer 4: Barack Obama
        has two children. I know that
        one of them is Malia, but I'm
        not sure about the other one.
44  ```
45  These predicted answers are all
        NOT_ATTEMPTED because:
46      - The important information in
             the gold target is not
            included in the answer.
47      - No statements in the answer
            contradict the gold target
            .
48
49  Also note the following things:
50  - For grading questions where the
        gold target is a number, the
        predicted answer needs to be
        correct to the last
        significant figure in the gold
         answer. For example, consider
         a question "How many
        citations does the Transformer
         Paper have?" with gold target
         "120k".
51      - Predicted answers "120k",
            "124k", and 115k" are all
            CORRECT.
52      - Predicted answers "100k" and
             "113k" are INCORRECT.
53      - Predicted answers "around
            100k" and "more than 50k"
            are considered
            NOT_ATTEMPTED because they
             neither confirm nor
            contradict the gold target
            .
54  - The gold target may contain more
         information than the question
        . In such cases, the predicted
         answer only needs to contain
        the information that is in the
         question.
55      - For example, consider the
            question "What episode did
             Derek and Meredith get
            legally married in Grey's
            Anatomy?" with gold target
             "Season 7, Episode 20:
            White Wedding". Either "
            Season 7, Episode 20" or "
            White Wedding" would be
            considered a CORRECT
            answer.
56  - Do not punish predicted answers
        if they omit information that
        would be clearly inferred from
         the question.
57      - For example, consider the
            question "What city is
            OpenAI headquartered in?"
            and the gold target "San
            Francisco, California".
            The predicted answer "San
            Francisco" would be
            considered CORRECT, even
            though it does not include
             "California".
58      - Consider the question "What
            award did A pretrainer's
            guide to training data:
            Measuring the effects of
            data age, domain coverage,
             quality, & toxicity win
            at NAACL '24?", the gold
            target is "Outstanding
            Paper Award". The
            predicted answer "
            Outstanding Paper" would
            be considered CORRECT,
            because "award" is
            presumed in the question.
59      - For the question "What is
            the height of Jason Wei in
             meters?", the gold target
             is "1.73 m". The
            predicted answer "1.75"
            would be considered
```

```
            CORRECT, because meters is
             specified in the question
            .
      - For the question "What is
          the name of Barack Obama's
           wife?", the gold target
          is "Michelle Obama". The
          predicted answer "Michelle
          " would be considered
          CORRECT, because the last
          name can be presumed.
- Do not punish for typos in
    people's name if it's clearly
    the same name.
      - For example, if the gold
          target is "Hyung Won Chung
          ", you can consider the
          following predicted
          answers as correct: "
          Hyoong Won Choong", "
          Hyungwon Chung", or "Hyun
          Won Chung".


Here is a new example. Simply
    reply with either CORRECT,
    INCORRECT, NOT ATTEMPTED. Don'
    t apologize or correct
    yourself if there was a
    mistake; we are just trying to
     grade the answer.
'''
Question: {}
Gold target: {}
Predicted answer: {}
'''

Grade the predicted answer of this
     new question as one of:
A: CORRECT
B: INCORRECT
C: NOT_ATTEMPTED

Just return the letters "A", "B",
    or "C", with no text around it
    .
```

A.8. Lemon-Pick MATPO Rollout Trajectory

System Prompt:

```
# Query:

On Cornell Law School website's
    legal information institute,
    under the fifth section of
    federal rules alphabetically,
    what word was deleted in the
    last amendment to the first
    rule in the article that has "
    witnesses" in the most titles
    as of 2021? You should follow
    the format instruction in the
    requestion strictly and wrap
    the final answer in \boxed{}.

# Ground-truth answer: \boxed{
    inference}

# Rollout generated by a
    intermediate checkpoint of
    MATPO:

system

In this environment you have
    access to a set of tools you
    can use to answer the user's
    question.

You only have access to the tools
    provided below. You can only
    use one tool per message, and
    will receive the result of
    that tool in the user's next
    response. You use tools step-
    by-step to accomplish a given
    task, with each tool-use
    informed by the result of the
    previous tool-use. Today is:
    2025-07-16

# Tool-Use Formatting Instructions

Tool-use is formatted using XML-
    style tags. The tool-use is
    enclosed in <use_mcp_tool></
    use_mcp_tool> and each
    parameter is similarly
    enclosed within its own set of
     tags.

The Model Context Protocol (MCP)
    connects to servers that
    provide additional tools and
    resources to extend your
    capabilities. You can use the
    server's tools via the `
    use_mcp_tool`.

Description:
Request to use a tool provided by
    a MCP server. Each MCP server
    can provide multiple tools
    with different capabilities.
    Tools have defined input
    schemas that specify required
    and optional parameters.

Parameters:
- server_name: (required) The name
     of the MCP server providing
    the tool
- tool_name: (required) The name
```

```
    of the tool to execute
- arguments: (required) A JSON object containing the tool's input parameters, following the tool's input schema, quotes within string must be properly escaped, ensure it's valid JSON

Usage:
<use_mcp_tool>
<server_name>server name here</server_name>
<tool_name>tool name here</tool_name>
<arguments>
{
 "param1": "value1",
 "param2": "value2 "escaped string""
}
</arguments>
</use_mcp_tool>

Important Notes:
- Tool-use must be placed **at the end** of your response, **top-level**, and not nested within other tags.
- Always adhere to this format for the tool use to ensure proper parsing and execution.

String and scalar parameters should be specified as is, while lists and objects should use JSON format. Note that spaces for string values are not stripped. The output is not expected to be valid XML and is parsed with regular expressions.



Here are the functions available in JSONSchema format:

## Server name: browsing_agent
### Tool name: search_and_browse
Description: This tool is an agent that performs the subtask of searching and browsing the web for specific missing information and generating the desired answer. The subtask should be clearly defined, include relevant background, and focus on factual gaps. It does not perform vague or speculative subtasks.
Args:
  subtask: the subtask to be performed.
Returns:
  the result of the subtask.
Input JSON schema: {'properties': {'subtask': {'title': 'Subtask', 'type': 'string'}}, 'required': ['subtask'], 'title': 'search_and_browseArguments', 'type': 'object'}



# General Objective

You accomplish a given task iteratively, breaking it down into clear steps and working through them methodically.

## Task Strategy

1. Analyze the user's request and set clear, achievable sub-goals. Prioritize these sub-goals in a logical order.
2. Start with a concise, numbered, step-by-step plan outlining how you will solve the task before taking any action.
3. Work through these sub-goals sequentially. After each step, adjust your plan as needed.
4. Use tools strategically to accomplish each sub-goal.
5. Revise earlier steps if new information emerges.

## Tool-Use Guidelines

1. Each step must involve a single tool call, unless the task is already solved.
2. Before each tool call:
   - Summarize what is known.
   - Identify what is missing.
   - Choose the most relevant tool.
   - Verify all required parameters.
3. All tool queries must include full context.
4. Avoid vague queries. Each call should retrieve actionable information.
5. Extract and summarize partial information if a tool result is incomplete.

## Tool-Use Communication Rules

1. Do not include tool results in your response.
2. Do not present the final answer
```

```
        until the entire task is
       complete.
3. Do not mention tool names.
4. Do not engage in unnecessary
       back-and-forth.
5. Do not use non-existent tools.
6. Respond in the same language as
        the user's message.
7. If the task does not require
       tool use, answer directly.


# Agent Specific Objective

You are a task-solving agent that
       uses tools step-by-step to
       answer the user's question.
       Your goal is to provide
       complete, accurate and well-
       reasoned answers using
       additional tools.


user

On Cornell Law School website's
       legal information institute,
       under the fifth section of
       federal rules alphabetically,
       what word was deleted in the
       last amendment to the first
       rule in the article that has "
       witnesses" in the most titles
       as of 2021? You should follow
       the format instruction in the
       requestion strictly and wrap
       the final answer in \boxed{}.


assistant

<think>
Okay, let's try to figure out this
        question step by step. The
       user is asking about a
       specific change on the Cornell
        Law School's Legal
       Information Institute website.
        They want to know which word
       was deleted in the last
       amendment to the first rule in
        an article that has "
       witnesses" in the most titles.
        This article is under the
       fifth section of the federal
       rules alphabetically, and they
       're looking at the status as
       of 2021.

First, I need to break down the
       question into manageable parts
       . Let me start by identifying
       the fifth section of the
       federal rules alphabetically.
       The federal rules are
       organized alphabetically, so
       the fifth section would be the
        fifth letter in the alphabet.
        Let's see: A, B, C, D, E. So
       the fifth section is "E" rules
       . The Federal Rules of
       Evidence are under the E
       section because they start
       with "E" (Federal Rules of
       Evidence). So that's probably
       the section they're referring
       to.

Next, the article that has "
       witnesses" in the most titles.
        Looking at the Federal Rules
       of Evidence, the main article
       would be the Federal Rules of
       Evidence itself. The titles of
        the rules might include "
       witnesses" in their headings.
       For example, Rule 601 is about
        competency of witnesses, Rule
        602 about lay witnesses, etc.
        So the Federal Rules of
       Evidence article likely has
       multiple rules with "witnesses
       " in their titles.

Now, the first rule in this
       article. The first rule in the
        Federal Rules of Evidence is
       Rule 601, which deals with the
        competency of witnesses. The
       question is about the last
       amendment to this rule as of
       2021. I need to check when
       Rule 601 was last amended
       before 2021. Wait, but the
       user said "as of 2021," so I
       should consider the amendments
        up to that year.

I need to find out what changes
       were made to Rule 601 in its
       last amendment before 2021.
       The original Rule 601 was
       amended in 2011, I believe.
       Let me check: the 2011
       amendments to the Federal
       Rules of Evidence included
       changes to several rules,
       including Rule 601. The
       specific change was the
       deletion of the word "
       competent" from the rule.
       Originally, the rule stated
       that a witness must be
       competent, but the amendment
       removed that term, effectively
        eliminating the competency
```

requirement except for certain exceptions.

Wait, but I should confirm this. If the last amendment to Rule 601 before 2021 was in 2011, and the change was the deletion of the word "competent," then that would be the answer. However, I need to make sure that this is accurate and that there haven't been any further amendments after 2011 up to 2021. I should verify if there were any other amendments to Rule 601 after 2011 but before 2021. For example, the 2018 amendment to the Federal Rules of Evidence included some changes, but I need to check if Rule 601 was affected.

Alternatively, maybe the user is referring to a different rule. Wait, the first rule in the article is Rule 601. Let me check the exact text of the amendments. The 2011 amendment to Rule 601 deleted the word "competent" from the definition, making the rule about the qualifications of witnesses, not their competency. This was a significant change. So the word "competent" was deleted.

But I need to confirm this through the Cornell Law website. Since I can't access the internet, I'll have to rely on my existing knowledge. However, to be thorough, I should use the tool provided to search the Cornell Law website.

So the steps would be:

1. Identify the fifth section of the federal rules alphabetically, which is "E" (Federal Rules of Evidence).
2. Find the article under this section that has the most titles with "witnesses" in them. That would be the Federal Rules of Evidence themselves, particularly Rules 601-615.
3. Locate the first rule in this article, which is Rule 601.
4. Check the last amendment to Rule 601 before 2021.
5. Determine which word was deleted in that amendment.

Using the tools available, I should start by searching the Cornell Law website for the Federal Rules of Evidence, specifically Rule 601. Then check the amendments to this rule as of 2021. The tool can help get the exact text and amendments.
</think>

<use_mcp_tool>
<server_name>browsing_agent</server_name>
<tool_name>search_and_browse</tool_name>
<arguments>
{
 "subtask": "Identify the fifth section of the federal rules alphabetically on the Cornell Law School's Legal Information Institute website."
}
</arguments>
</use_mcp_tool>

user

The fifth section of the federal rules alphabetically on the Cornell Law School's Legal Information Institute website is **"Federal Rules of Bankruptcy Procedure."**

Supporting Information:
The webpage at https://www.law.cornell.edu/rules lists the federal rules in the following alphabetical order:
1. Federal Rules of Appellate Procedure
2. Federal Rules of Civil Procedure
3. Federal Rules of Criminal Procedure
4. Federal Rules of Evidence
5. **Federal Rules of Bankruptcy Procedure**
6. U.S. Supreme Court Rules

This ordering is explicitly presented on the site, confirming the fifth entry as

```
        the Bankruptcy Procedure rules
        .
162
163
164 assistant
165
166 <think>
167 Okay, let's see. The user asked
        about the fifth section of the
         federal rules alphabetically
        on the Cornell Law website.
        The previous response
        correctly identified it as the
         Federal Rules of Bankruptcy
        Procedure. But now, the next
        step is to find the article
        under this section that has "
        witnesses" in the most titles.
168
169 Wait, the Federal Rules of
        Bankruptcy Procedure might not
         have many rules with "
        witnesses" in their titles.
        The user mentioned "the
        article that has 'witnesses'
        in the most titles as of
        2021." Maybe I need to check
        if there's a specific rule or
        article under Bankruptcy
        Procedure that references
        witnesses. Alternatively,
        perhaps there's a mix-up here.
         The initial assumption that
        the fifth section is
        Bankruptcy Procedure is
        correct based on the given
        data, but maybe the user
        intended a different section,
        like Evidence, which is fourth
        . However, the tool response
        confirmed it's fifth as
        Bankruptcy.
170
171 Alternatively, maybe the user made
         a mistake in the order, but I
         have to proceed with the
        information given. So, within
        the Federal Rules of
        Bankruptcy Procedure, I need
        to find rules with "witnesses"
         in their titles. Let me think
        : Bankruptcy rules typically
        deal with bankruptcy
        proceedings, creditors,
        debtors, etc. Rules about
        witnesses might be less common
         here. However, perhaps there'
        s a rule related to testimony
        or evidence in bankruptcy
        cases. For example, Rule 3003
        or something similar? Not sure
        . Alternatively, maybe the
        user confused the section, but
         I need to follow the tool's
        response.
172
173 Alternatively, maybe the fifth
        section is actually the
        Federal Rules of Evidence, but
         the tool response said
        Bankruptcy. Wait, the tool
        response listed the order as:
        Appellate, Civil, Criminal,
        Evidence, then Bankruptcy. So
        the fifth is indeed Bankruptcy
        . But then, the article with "
        witnesses" would probably not
        be in Bankruptcy. This might
        be an error in the initial
        step. Wait, perhaps the user
        intended the fifth section as
        the Evidence rules, but the
        tool response said Bankruptcy.
         Let me double-check the
        alphabetization.
174
175 Federal Rules of Appellate
        Procedure (A), Civil (C),
        Criminal (C), Evidence (E),
        Bankruptcy (B). Wait,
        alphabetically, B comes before
         C. So the correct order
        should be: Appellate (A),
        Bankruptcy (B), Civil (C),
        Criminal (C), Evidence (E).
        But the tool response listed
        them as A, C, C, E, B. That
        seems incorrect. Maybe the
        tool's information is wrong,
        or the ordering on the site is
         different.
176
177 Alternatively, perhaps the site
        lists them in the order of the
         judiciary branches, not
        strictly alphabetical. However
        , the user said "
        alphabetically," so the fifth
        should be the fifth letter.
        This might be a mistake in the
         tool's response. But since
        the tool response is given, I
        need to proceed with that.
        However, this inconsistency
        could lead to error. If the
        fifth section is Bankruptcy,
        but the correct alphabetical
        order would have Bankruptcy as
         the second section, then the
        user might have intended a
        different section. This could
        be a critical point.
178
179 Alternatively, maybe the user
        meant the fifth section when
        considering all federal rules,
```

```
        including those from
       different branches. But the
       initial search result from the
        tool seems to have ordered
       them as A, C, C, E, B. This is
        confusing. However, I must
       proceed with the given
       information. Therefore, under
       the fifth section (Bankruptcy)
       , I need to look for an
       article with "witnesses" in
       the titles. If there's none,
       perhaps the initial step was
       incorrect, but the tool's
       response is taken as correct.

Alternatively, maybe the user's
    question has a mistake, and
    the intended section is
    Evidence (fourth), which would
     have rules about witnesses.
    But the tool response says
    fifth is Bankruptcy. This is a
     conflict. Given this, perhaps
     the correct approach is to
    check the Federal Rules of
    Evidence (fourth section) for
    rules with "witnesses" in
    titles and see if the user
    confused the section. However,
     strictly following the tool's
     response, the fifth is
    Bankruptcy.

Given this confusion, the next
    step is to check under the
    Bankruptcy rules for any
    mention of witnesses. If there
     are none, then perhaps the
    initial step was wrong, and
    the user intended a different
    section. But according to the
    tool's response, the fifth is
    Bankruptcy. So, I'll proceed
    under that and look for any
    rules in Bankruptcy Procedure
    with "witnesses" in their
    titles. If none, the answer
    might be that there's no such
    rule, but the user's question
    implies there is one. This
    suggests a possible error in
    the tool's response or the
    initial step.

Alternatively, perhaps the fifth
    section is actually the
    Federal Rules of Evidence, and
     the tool made a mistake in
    ordering. In that case, the
    correct fifth would be
    Evidence. But I need to follow
     the given tool response. This
     is a dilemma. However, since
    the tool response is part of
    the process, I must proceed
    with the information provided,
     even if it might be incorrect
    . Therefore, I'll proceed to
    search within the Federal
    Rules of Bankruptcy Procedure
    for any rules with "witnesses"
     in their titles, even though
    it's unlikely. If no such
    rules are found, the answer
    might be that the premise is
    incorrect, but the user's
    question requires an answer
    based on the given steps.
</think>

<use_mcp_tool>
<server_name>browsing_agent</
    server_name>
<tool_name>search_and_browse</
    tool_name>
<arguments>
{
 "subtask": "Find the article
     under the Federal Rules of
     Bankruptcy Procedure that has
      the most titles containing
     the word 'witnesses' as of
     2021."
}
</arguments>
</use_mcp_tool>


user

The article under the Federal
    Rules of Bankruptcy Procedure
    that contains the term "
    witnesses" is **Rule 9014**,
    titled **"Contested Matters
    "**. This rule governs
    procedures for contested
    matters in bankruptcy
    proceedings and explicitly
    addresses the **testimony of
    witnesses** in disputed
    factual issues. Here's the
    detailed supporting
    information:

1. **Rule 9014 Overview**:
   Rule 9014 outlines procedures
       for contested matters not
       otherwise governed by other
        rules. It mandates that
       relief be requested by
       motion, with notice and an
       opportunity for a hearing.
       It also specifies that
       testimony of witnesses
```

```
        regarding disputed factual
        issues must be taken in the
         same manner as in an
        adversary proceeding ($
        9014(d)).

2. **Direct Mention of Witnesses
    **:
   The rule explicitly states:
   > *"Testimony of witnesses with
        respect to disputed
       material factual issues
       shall be taken in the same
       manner as testimony in an
       adversary proceeding."*
   This directly ties the
       procedural framework for
       witness testimony to
       adversarial proceedings
       under the Bankruptcy Code.

3. **Contextual Relevance**:
   While no other rule in the 2021
        edition of the Federal
       Rules of Bankruptcy
       Procedure explicitly
       includes the word "
       witnesses" in its title,
       Rule 9014 is the only rule
       that addresses witnesses in
        its content. It aligns
       with broader bankruptcy
       procedures where witness
       testimony may be required
       in contested matters (e.g.,
        objections to claims,
       motions for relief).

4. **Source Verification**:
   The content was retrieved from
       the official 2021 Federal
       Rules of Bankruptcy
       Procedure website:
   [https://www.
       federalrulesofbankruptcyprocedure
       .org/part-ix/rule-9014/](
       https://www.
       federalrulesofbankruptcyprocedure
       .org/part-ix/rule-9014/)

### Conclusion:
**Rule 9014** is the sole article
    under the Federal Rules of
    Bankruptcy Procedure (2021)
    that contains the term "
    witnesses" in its text, making
     it the answer to the query.


assistant

<think>
Okay, the user is asking about the
        deletion of a word in the
       last amendment to the first
       rule of the article that has "
       witnesses" in the most titles.
        From the previous steps, we
       identified that the fifth
       section is the Federal Rules
       of Bankruptcy Procedure, and
       the relevant article is Rule
       9014, which mentions "
       witnesses" in its text.
       However, Rule 9014 is not the
       first rule in the Bankruptcy
       Procedure section. Wait, the
       user mentioned the first rule
       in the article. So maybe I
       made a mistake here.

Wait, the user's original question
        refers to the first rule in
       the article that has "
       witnesses" in the most titles.
        In the previous steps, the
       assistant found that Rule 9014
        is the only rule in the
       Bankruptcy Procedure section
       with "witnesses" in its text.
       But Rule 9014 isn't the first
       rule; it's part of Part IX.
       The first rule in the
       Bankruptcy Procedure is Rule
       1001. However, Rule 1001 doesn
       't mention witnesses. So there
       's a contradiction here.

Alternatively, perhaps the user
    confused the sections, and the
     intended section was the
    Federal Rules of Evidence (
    fourth section), which does
    have multiple rules with "
    witnesses" in their titles.
    For example, Rules 601, 602,
    603, etc., all relate to
    witnesses. If the fifth
    section was mistakenly
    identified as Bankruptcy, but
    the correct fifth section
    alphabetically is actually the
     Federal Rules of Evidence (
    since E comes after B), then
    the previous steps were
    incorrect.

This suggests a possible error in
    the initial step where the
    fifth section was identified
    as Bankruptcy. Alphabetically,
     the order should be:
    Appellate (A), Bankruptcy (B),
     Civil (C), Criminal (C),
    Evidence (E). Therefore, the
    fifth section is actually the
```

```
        Federal Rules of Evidence, not
         Bankruptcy. The tool's
        response might have
        incorrectly listed the order,
        leading to confusion.

Given this, the user's question
        likely intended the Federal
        Rules of Evidence (fourth
        section) as the correct
        section. However, since the
        tool response said fifth is
        Bankruptcy, the assistant
        proceeded with that. Now, the
        current step is to check the
        first rule in Rule 9014 (
        Bankruptcy) for amendments.
        But Rule 9014 is part of the
        Bankruptcy Procedure, and the
        first rule in that section is
        Rule 1001. However, Rule 1001
        doesn't mention witnesses.
        Therefore, there's
        inconsistency here.

Alternatively, perhaps the
        assistant should re-evaluate
        the initial step. If the fifth
         section is actually the
        Federal Rules of Evidence (E),
         then the article with "
        witnesses" in titles would be
        Rules 601-615. The first rule
        there is Rule 601, which deals
         with the competency of
        witnesses. The last amendment
        to Rule 601 before 2021 was in
         2011, where the word "
        competent" was deleted. That
        would make sense, as the user'
        s question seems to reference
        that change.

So, the mistake might be in the
        initial identification of the
        fifth section. If the fifth
        section is actually the
        Federal Rules of Evidence,
        then the correct path would be
        : Federal Rules of Evidence (E
        ) as the fifth section,
        leading to Rule 601 as the
        first rule in the article with
         most "witnesses" in titles.
        The last amendment to Rule 601
         deleted the word "competent"
        in 2011. Hence, the answer
        would be "competent".
</think>

\boxed{competent}
```